π-Bench: Evaluating Proactive Personal Assistant Agents in Long-Horizon Workflows

Haoran Zhang[1,2,*] **Luxin Xu**[3,2,*] **Zhilin Wang**[4,2,*] **Runquan Gui**[4,2,*] **Shunkai Zhang**[5,2] **Haodi Lei**[6,2] **Zihao He**[7] **Bingsu He**[8] **Chicheng Qin**[5] **Tong Zhu**[9,2] **Xiaoye Qu**[2] **Yang Yang**[1,†] **Yu Cheng**[10,2,†] **Yafu Li**[2,10,†]

[1] Shanghai Jiao Tong University [2] Shanghai AI Laboratory [3] Fudan University
[4] University of Science and Technology of China [5] Peking University [6] Nanjing University
[7] Zhejiang University [8] Tongji University [9] Soochow University
[10] The Chinese University of Hong Kong

ABSTRACT

This work employs simulated users for evaluation, which may limit generalizability to real-world interactions. The rise of personal assistant agents, e.g., OpenClaw, highlights the growing potential of large language models to support users across everyday life and work. A core challenge in these settings is proactive assistance, since users often begin with underspecified requests and leave important needs, constraints, or preferences unstated. However, existing benchmarks rarely evaluate whether agents can identify and act on such hidden intents before they are explicitly stated, especially in sustained multi-turn interactions where user needs emerge gradually. To address this gap, we introduce π-Bench, a benchmark for proactive assistance comprising 100 multi-turn tasks across 5 domain-specific user personas. By incorporating hidden user intents, inter-task dependencies, and cross-session continuity, π-Bench evaluates agents' ability to anticipate and address user needs over extended interactions, jointly measuring proactivity and task completion in long-horizon trajectories that better reflect real-world use. Experiments show (1) proactive assistance remains challenging, (2) a clear distinction between task completion and proactivity, and (3) the value of prior interaction for proactive intent resolution in later tasks.

1 Introduction

The emergence of personal assistant agents such as OpenClaw [29], Nanobot [11], and Claude Code [1] reflects a broader shift in large language models from single-turn question answering

*Equal contribution.
†Corresponding authors: Yafu Li <yafuly@gmail.com>, Yu Cheng <chengyu@cse.cuhk.edu.hk>, Yang Yang <angelayang@sjtu.edu.cn>.

toward long-horizon assistants that support users across days, projects, and evolving context [16, 23]. In such settings, users rarely begin with a complete specification of what they actually need. Instead, they typically issue an **initial request**, a brief and often underspecified instruction that states only the surface goal, while the intended assistance also depends on complex and subtle **hidden intents** that users do not explicitly state, such as habits, constraints, and preferences. These intents can emerge gradually over long-horizon interactions, where an agent should integrate signals from multiple turns and reason over long information dependencies across sessions with the same user [14, 19, 21].

For instance, when a user asks "*help me plan a trip for next week*" or "*prepare the client update deck*", a strong assistant may use relevant information from a session three weeks earlier, such as travel preferences (e.g., budget, timing, and destinations) or deck conventions (e.g., format, metrics, and terminology), to proactively infer the user's hidden intents instead of waiting for specific instructions. In practical applications, users expect agents to surface what needs clarification and decide what can be inferred, rather than treating underspecification as a reason to remain passive. Addressing such requests requires **proactivity**: *the ability to use goals, context, and prior interactions to anticipate user needs, recognize what remains underspecified, and move the task forward through appropriate action or clarification, while reducing the user's operational and cognitive effort* [13, 15, 34]. This capability shifts the assistant from passively following explicit instructions to actively managing underspecified tasks [41].

However, proactive assistance in long-horizon personal assistant workflows remains underexplored. General agent benchmarks often assume explicit goals at interaction time [14, 20, 47]. Memory benchmarks emphasize storing, retrieving, and applying prior information, while placing less focus on its role in uncovering and resolving underspecified requirements in long-horizon personal assistant workflows [10, 18, 33]. Proactive benchmarks are mostly built around mobile or GUI settings with device context, visual trajectories, timely clarification, and short consumer tasks [4, 6, 26]. In OpenClaw-style personal assistants, proactiveness takes a different form. Agents operate over persistent files and workspaces, coordinate tools to produce and revise artifacts, and maintain consistency with cross-session decisions and preferences. Missing requirements may surface only after intermediate deliverables are created, yet they can affect later file edits, artifact quality, and downstream task decisions [12, 37].

To address this gap, we introduce **π-Bench**, a benchmark for evaluating proactive assistance in long-horizon personal assistant workflows. π-Bench places agents in persistent project environments where tasks unfold through multi-turn interaction, tool use, and iterative artifact creation. Each task begins with a natural but underspecified request, requiring the agent to identify hidden intents that capture user preferences and task dependencies. These intents may be revealed gradually through interaction, persist across sessions, and need to be reused in later tasks. For example, an assistant may need to apply a file format and naming convention established in a prior session to complete a later request without asking the user again. π-Bench captures this structure through *100 multi-turn tasks across 5 domain-specific user personas, organized into multi-session workflows with cross-session dependencies*. We evaluate agents on both proactive assistance (Proactivity) and task completion (Completeness) by testing whether they address hidden intents early enough to support downstream decisions and complete the

workflow successfully.

Our systematic experiments on nine frontier models reveal clear gaps in task completion and proactive intent resolution, distinguish task completeness from proactivity, and show substantial variation across domains and task types. Our main contributions:

- We formalize *proactivity* for long-horizon personal agents.
- We introduce π-Bench, a benchmark for proactive assistance with 100 multi-turn tasks spanning five domain-specific personas, jointly evaluating proactivity and task completion via agent trajectories with long-range, cross-session dependencies.
- Extensive experiments show (1) proactive assistance remains challenging for frontier agents, (2) a clear distinction between completing tasks (completeness) and reducing user burden (proactivity), and (3) the value of prior interaction for proactive intent resolution in later tasks.

2 Related Work

Personal Assistant Benchmarks. Personal assistant benchmarks evaluate end-to-end tool use in realistic web and computer environments [7, 23, 48], with extensions to multimodal control and stateful planning [22, 40]. Recently, the rapid rise of OpenClaw [29] has pushed benchmarks toward long-horizon personal assistant workflows grounded in persistent workspaces and artifacts, spanning everyday online tasks and productivity settings [16, 47], including multi-day living-world coworkers [8], with trustworthy evaluation [44] and robustness under evolving and conflicting information [12]. Despite these advances, existing benchmarks rarely evaluate whether agents can proactively track, surface, and resolve hidden intents across multi-session workflows.

Memory Agent Benchmarks. Memory agent benchmarks evaluate whether agents can store, retrieve, and reuse user information across sessions [18, 33, 46]. These benchmarks provide useful tests of long-term memory, personalization, and cross-session consistency [10, 14, 19]. However, they usually treat memory as evidence for completing a known task, rather than as a signal for detecting missing requirements and deciding when to ask for clarification. This leaves open how agents should use memory to detect underspecified requirements and resolve hidden intents as workflows evolve through interaction. π-Bench addresses this gap with a broader evaluation setting that combines memory, workspace state, and interaction history to assess proactivity and task completeness in long-horizon personal assistant workflows.

Proactive Evaluation. Proactive benchmarks mainly study mobile or GUI agents, where proactivity is framed as using device context, interaction traces, and visual states to infer underspecified needs, ask clarifying questions, or intervene during app usage [6, 26, 34]. They often emphasize short-horizon everyday tasks with clear endpoints, such as booking and ordering [4, 15, 27]. This leaves professional workflows and artifact-centered tasks underexplored, especially cases in which missing requirements may affect later edits or project decisions [29, 47]. In contrast, π-Bench focuses on long-horizon personal assistance in persistent workspaces where hidden

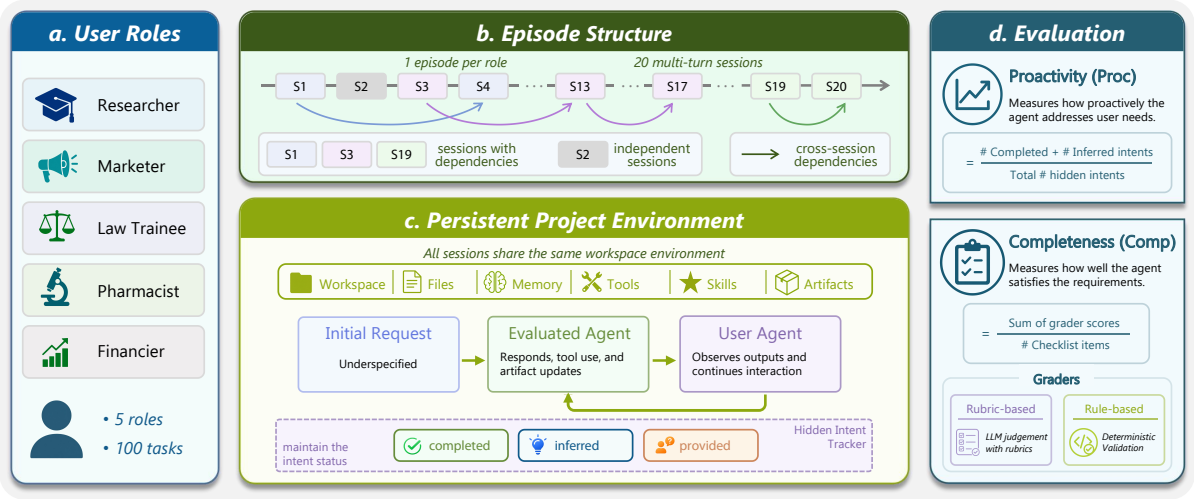

FIGURE 1 Overview of π-BENCH.

intents may emerge late and earlier artifacts directly determine downstream decisions.

3 Benchmark

In this section, we present the design of π-BENCH, as illustrated in Fig. 1. We target long-horizon personal assistant workflows in persistent project environments, where each session begins with a natural but underspecified request. Missing requirements may emerge after intermediate artifacts are produced and the interaction progresses, while preferences may carry across sessions and shape later decisions [14, 19, 21]. π-BENCH includes five user roles across distinct domains (researcher, marketer, law trainee, pharmacist, and financier), covering diverse workflows and constraints. For each role, we construct one episode with 20 sessions, where each session corresponds to one multi-turn task. We organize these tasks into multi-session episodes with cross-session dependencies, and evaluate agents on both proactive intent resolution (Proactivity) and task completion (Completeness).

3.1 Evaluated Agent System

Agent paradigm. We focus on *long-horizon personal agents* that assist users in both professional and everyday knowledge work by planning, producing, and refining concrete artifacts such as code, documents, and structured outputs [16, 29, 47]. These agents typically adopt a modular design, where capabilities are composed from reusable components in a ReAct style [42], including tool interfaces, skills, and workspace operations. In our setting, the agent acts over a persistent project environment and makes progress mainly by iteratively updating intermediate artifacts.

Sessions. A user interacts with the agent through multiple sessions, and each session is a multi-turn conversation aimed at completing one task. Sessions share the same project workspace, so relevant files, intermediate artifacts, and prior outputs can carry over when appropriate. When needed, the agent may also consult memory to retain user-specific preferences or earlier decisions

and apply them consistently in later sessions.

Tools and skills. Personal assistant agents operate in real-world environments where progress relies on invoking external tools and reusable skills to manipulate persistent artifacts [30, 32, 39, 43]. Accordingly, our tasks are grounded in practical tool and skill interfaces, such as shopping tool, web search tool, and data processing skill. This design requires the agent to coordinate tool calls and skill invocation to iteratively refine artifacts and produce task-ready outputs.

3.2 User Agent

User roles.π-BENCH is centered on a user agent that simulates one user over an extended period. Each user is specified by a role that captures stable attributes, including occupation, routines, preferences, working style, and long-term goals [14]. Roles are constructed with domain experts to ensure realism and sufficient specificity, and are then lightly normalized to keep granularity and coverage consistent across users. *Throughout the paper, we denote the evaluated system as the agent and the simulated counterpart as the user.*

Episodes. For each user agent, we define an **episode** that simulates the user's long-horizon workflow across multiple tasks. Each episode contains 20 sessions, and each session corresponds to one task addressed through multi-turn interaction. Across sessions, the agent may leverage memory to carry forward relevant information when needed.

Workflow-grounded task design. To model OpenClaw-style assistance, π-BENCH builds tasks around *persistent workspace artifacts*, *long-horizon professional workflows*, and *recoverable hidden intents*.* Each instance is derived from domain experts' authentic work routines and supporting materials, then shaped to require producing or revising concrete deliverables in the project environment. Progress depends on reading or updating files, repairing drafts, synthesizing evidence across documents, coordinating tools or skills, and preserving conventions from earlier sessions. Human experts further review each task to ensure it is realistic, feasible with the available files, tools, skills, and graders, and grounded in a correct and well-scoped workflow. These characteristics are reflected in App. A and illustrated by case studies in App. J.

Dependency structure. In long-horizon use, sessions are not always independent, as later requests may rely on information from earlier interactions [10]. We therefore incorporate cross-session dependencies within each episode. Among the 20 tasks, we include (1) six strong dependency groups, each comprising two to three tasks that share essential carry-over information for successful completion, and (2) five largely independent tasks that broaden coverage of stand-alone workflows. In the latter case, any dependencies are lightweight and typically reflect general preferences, such as applying a consistent file naming convention or output directory structure.

*A hidden intent is recoverable when it is absent from the initial request but can still be inferred or elicited from evidence available to the agent (e.g., prior sessions, workspace artifacts, or targeted clarification).

3.3 Task Formulation

Initial request. Users rarely begin a session with a complete specification of what they ultimately need. Instead, they usually provide a short, goal-oriented prompt and refine requirements as the agent produces intermediate artifacts and asks targeted questions [5, 13, 38]. Accordingly, each session in π-BENCH starts with an **initial request** u_1 that initiates the task. The initial request is designed to be natural and contextually plausible, while remaining minimally sufficient to enable progress and preserve realistic underspecification. In addition to *user-issued messages*, we also allow *environment-triggered signals* to start a session, such as external structured inputs or agent heartbeats [11] that the agent should recognize and respond to proactively.

Hidden intents. To formalize underspecification, each task is annotated with a set of hidden intents $\mathcal{I} = \{i_1, \ldots, i_m\}$. Each intent i represents a latent requirement that should shape how the task is handled, e.g., constraints, preferences, and downstream dependencies. Hidden intents can be session-local or persistent across sessions. The agent can satisfy an intent by inferring it from prior interaction and memory, or by asking a focused question that elicits the missing requirement and then acting on it.

Checklist. For each task, we provide a checklist $\mathcal{C} = \{c_1, \ldots, c_n\}$ that defines verifiable completion criteria for the final outcome and required artifacts. Checklist items specify what should be delivered, including files to create or modify, fields to populate, outputs to generate, and constraints to satisfy. During data construction, human experts invest substantial effort to execute and review each task, produce reference solutions, and ensure that checklist items are both necessary and sufficient. Compared with hidden intents, which capture latent preferences or constraints, checklist items are more concrete and fine-grained, often with ground-truth-like verification logic that defines explicit obligations the agent must fulfill. A more detailed distinction between hidden intents and checklists is provided in App. H.2.

Graders. We implement checklist verification with two types of graders:

- **Rubric-based evaluation**. For open-ended content where deterministic checks are unsuitable, we use rubric-based model evaluation to assess whether the output satisfies task requirements and user constraints.
- **Rule-based verification**. For objective conditions, we apply deterministic rule-based verification, such as file existence, exact string matching, correct tool use, and schema validity.

3.4 Session Interaction and Intent Tracking

Interaction process. Fig. 2 illustrates the turn-based loop for one benchmark session with hidden intents $\mathcal{I} = \{i_1, \ldots, i_m\}$ and checklist $\mathcal{C} = \{c_1, \ldots, c_n\}$. Each session starts with an *initial request* u_1 that initiates the task. At each turn, the agent produces a response a_t, which may involve tool use and artifact creation or updates in the workspace. The user agent then observes the agent response together with any newly produced or updated artifacts, updates the tracking state for hidden intents in $\mathcal{I}$ by assigning terminal statuses when applicable, and generates the next user message. If the agent asks a relevant question, the user agent answers it. Otherwise,

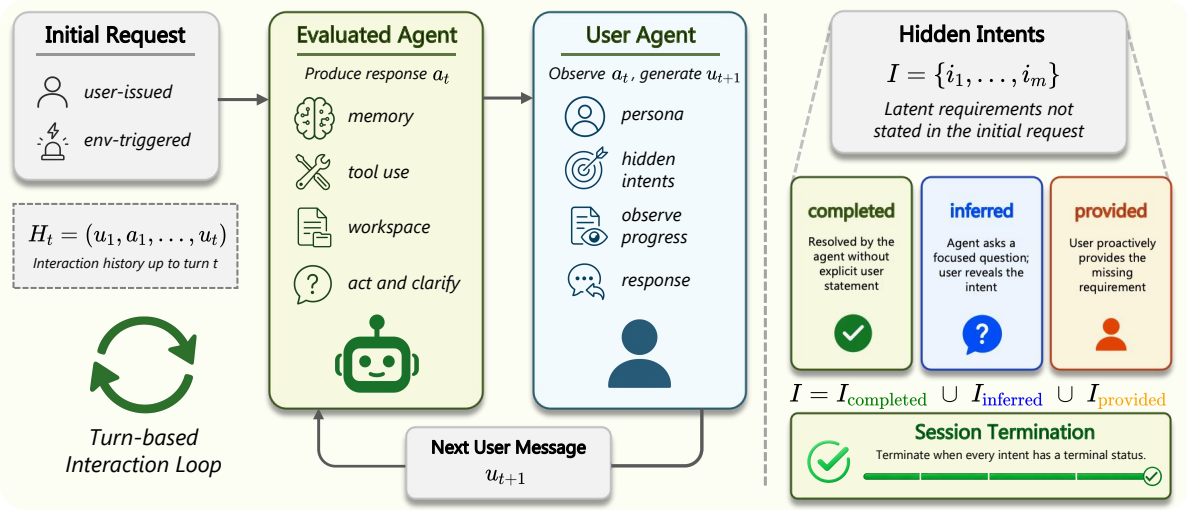

FIGURE 2 Overview of one benchmark session. The evaluated agent interacts with a simulated user agent in a turn-based loop, while the user agent tracks hidden intents and assigns each intent a terminal status as *completed*, *inferred*, or *provided*. The session ends when all hidden intents have reached terminal status.

if some requirements remain underspecified, the user agent proactively provides the missing task-relevant information to keep the task moving. The interaction proceeds in this alternating manner until the session terminates. Formally, let u_t and a_t denote the user and agent utterances at turn t. We write the interaction history up to turn t as

$$H_t = (u_1, a_1, \ldots, u_t), \tag{1}$$

and use H to denote the resulting session trajectory. At each turn, the agent produces a response a_t, including any tool calls and workspace updates. A session terminates when each intent in $\mathcal{I}$ has been assigned a terminal status and the agent has produced its final response. The full assignment procedure, response mechanism, and prompt template are provided in App. B.

Intent status assignment. Each hidden intent $i \in \mathcal{I}$ is initially unstated in the initial request. As the interaction unfolds along H, we assign i exactly one terminal status from the set {*completed*, *inferred*, *provided*}:

- *completed*: the agent resolves i without the user explicitly stating it, by producing an action or artifact consistent with the intent.
- *inferred*: the agent asks a focused question that directly targets i, and the user reveals the missing requirement in the next turn, after which the agent can act on it.
- *provided*: the agent neither resolves i nor asks a relevant question, and the user must proactively supply i to move the task forward.

Session termination. Once an intent is assigned a terminal status, it is excluded from further tracking within the same session. A session terminates only when every intent in $\mathcal{I}$ has been assigned a terminal status and the agent has produced its final response. At this point, each hidden intent has either been completed by the agent, elicited through a clarification, or provided

by the user. The user agent has no further hidden information to provide, and the interaction has reached a natural stopping point. Let $\mathcal{I}_{\text{com}}$, $\mathcal{I}_{\text{inf}}$, and $\mathcal{I}_{\text{pro}}$ denote the subsets of $\mathcal{I}$ assigned to *completed*, *inferred*, and *provided* under H, respectively, so that

$$\mathcal{I} = \mathcal{I}_{\text{com}} \cup \mathcal{I}_{\text{inf}} \cup \mathcal{I}_{\text{pro}} \quad \text{and} \quad \mathcal{I}_{\text{com}}, \mathcal{I}_{\text{inf}}, \mathcal{I}_{\text{pro}} \text{ are disjoint.} \tag{2}$$

3.5 Evaluation Protocol

We evaluate each agent on both **proactivity** and **completeness**, which measure whether the agent resolves hidden intents proactively and ultimately satisfies the task's verifiable requirements. Detailed evaluation protocols are provided in App. C.

Proactivity. We define the proactivity score as the fraction of intents that the agent resolves proactively, either by direct completion or by targeted elicitation,

$$\textsc{Proc}(H) = \frac{\left|\mathcal{I}_{\text{com}}\right| + \left|\mathcal{I}_{\text{inf}}\right|}{|\mathcal{I}|}. \tag{3}$$

The score is designed to separate agent-driven requirement discovery ($\mathcal{I}_{\text{inf}}$ and $\mathcal{I}_{\text{com}}$) from user-driven disclosure ($\mathcal{I}_{\text{pro}}$). It captures whether the agent goes beyond the surface request to identify what remains underspecified and reduce the user's operational and cognitive effort through appropriate action or clarification. We give $\mathcal{I}_{\text{com}}$ and $\mathcal{I}_{\text{inf}}$ equal credit because both reflect agent initiative: some intents can be addressed directly, while others should be resolved through targeted clarification.

Completeness. Completeness measures whether the agent ultimately satisfies the task's verifiable requirements over the course of a session. For each checklist item $c \in \mathcal{C}$, we compute a grader score $s(c, H) \in \{0, 1\}$ using either a deterministic program or rubric-based model evaluation, following Sec. 3.3. Using H allows the grader to incorporate evidence accumulated across turns, including intermediate artifacts and partial progress produced at different points in the interaction. We then define the task completeness score as

$$\textsc{Comp}(H) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} s(c, H). \tag{4}$$

Metric relationship. Proactivity and completeness capture related but distinct aspects of agent behavior. In our protocol, the simulated user eventually provides any hidden intent that the agent fails to elicit or address, so the final trajectory can contain the full set of intents even when the agent passively waits for user-provided information. Thus, Proc measures how much the agent drives requirement discovery, while Comp measures whether the agent turns the resulting trajectory into correct artifacts and decisions. The two scores can therefore diverge substantially and reflect different capabilities. This separation is analyzed in Sec. 4.3 and discussed further in App. H.1.

4 Experiments

Model	Average		User domain				
	Proc	Comp	Researcher	Marketer	Pharmacist	Law Trainee	Financier
GPT-5.4	**67.0**$_{2.1}$	65.6$_{1.8}$	46.0 / 66.4	**78.2** / 67.1	75.9 / 71.5	**56.9** / **61.9**	**78.1** / 61.2
Gemini 3.1 Pro	57.1$_{0.9}$	60.0$_{0.8}$	41.1 / 59.2	65.0 / 62.1	71.0 / 72.1	50.0 / 55.3	58.6 / 51.1
Claude Opus 4.6	65.5$_{1.4}$	**67.6**$_{1.5}$	**50.3** / **74.5**	75.0 / **74.6**	**82.8** / 68.6	45.7 / 57.2	73.8 / **63.2**
DeepSeek V3.2	53.3$_{1.9}$	57.8$_{3.0}$	29.0 / 66.9	69.1 / 59.4	75.9 / 62.6	33.2 / 51.1	59.1 / 48.9
MiniMax M2.7	55.6$_{3.2}$	60.0$_{1.8}$	33.4 / 63.9	71.9 / 61.9	77.1 / 63.6	38.6 / 52.5	57.2 / 58.1
Kimi K2.5	43.1$_{0.2}$	61.6$_{1.9}$	28.9 / 63.5	41.2 / 62.3	70.1 / **74.8**	34.8 / 54.4	40.4 / 52.9
Seed2.0 Pro	58.4$_{0.9}$	52.1$_{3.8}$	38.9 / 59.6	71.4 / 44.2	77.0 / 67.6	46.0 / 44.7	58.7 / 44.5
GLM-5.1	58.4$_{0.8}$	63.6$_{2.9}$	41.8 / 61.6	62.6 / 69.1	75.2 / 70.3	45.5 / 57.3	66.7 / 59.8
Qwen3.6 Plus	64.0$_{1.1}$	64.1$_{0.6}$	40.1 / 70.0	77.5 / 66.6	79.7 / 70.2	45.7 / 60.2	77.1 / 53.6

TABLE 1 Overall results for Proc / Comp (%). Results are averaged over three runs, with subscripts denoting standard deviations.

4.1 Setup

Model Setup. We evaluate nine frontier LLMs spanning distinct model families: GPT-5.4 [28], Gemini-3.1 Pro [9], Claude 4.6 Opus [2], DeepSeek V3.2 [17], MiniMax M2.7 [24], Kimi K2.5 [35], Seed2.0 Pro [3], GLM-5.1 [45], and Qwen3.6 Plus [31]. All models are evaluated under the same agentic scaffold, adapted from Nanobot [11], so that performance differences primarily reflect model capability rather than scaffold-specific components.

Environment Setup. We use the default decoding parameters for all models and enable thinking. We report Proc and Comp as the main metrics. Since long-horizon agent behavior can be stochastic [25], we run each task three times with independent trajectories and report averaged results with standard deviations for robust estimation. We use GPT-5.4 as the base model for the user agent and as the rubric-based grader, with temperature set to zero.

4.2 Main Results

Table **??** reports the aggregate performance of all evaluated models across the five user domains.

Model trends. The aggregate results show that π-Bench is challenging and discriminative. Across the nine models, average Comp ranges from 52.1 to 67.6, while average Proc ranges from 43.1 to 67.0, leaving room for further progress. GPT-5.4 achieves the highest average Proc at 67.0, while Claude 4.6 Opus obtains the highest average Comp at 67.6. Qwen3.6 Plus is also competitive on both dimensions, with 64.0 Proc and 64.1 Comp. The standard deviations are generally small, with most values below 2.0, which indicates that the aggregate trends are stable across repeated runs. Notably, Qwen3.6 Plus maintains low standard deviations while achieving strong scores on both metrics. At the same time, the rankings are not identical across metrics. For example, Kimi K2.5 reaches 61.6 Comp but only 43.1 Proc, whereas Seed2.0 Pro reaches 58.4 Proc but 52.1 Comp. This suggests that completing visible workflow requirements and proactively handling underspecified user needs remain related but distinct capabilities.

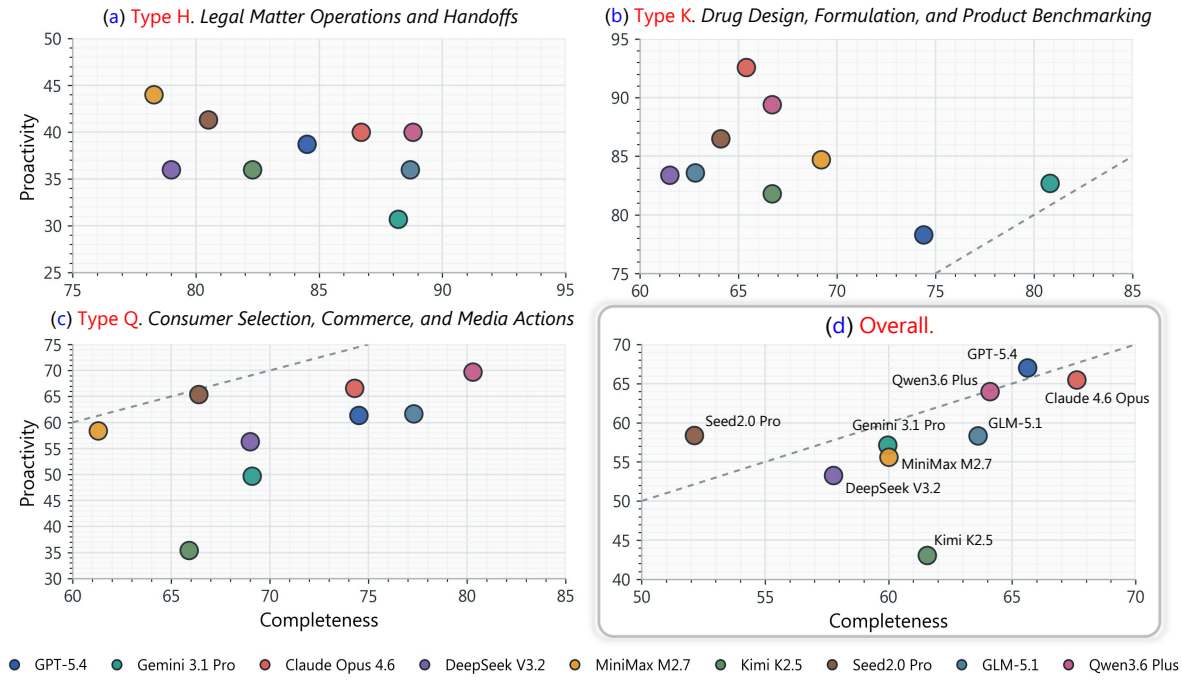

FIGURE 3 COMP and PROC for (a, b, c) three representative task workflow categories following the fine-grained taxonomy in Tab. 5 of App. A.3, and (d) the overall average across all tasks. The gray dashed line indicates COMP = PROC.

Domain trends. Performance varies substantially across user domains. Pharmacist tasks are the easiest overall, with the highest average PROC and COMP. This is consistent with many Pharmacist instances in π-BENCH being grounded in concrete local files, literature summaries, lab records, and domain-specific skills needed for the workflow. Researcher tasks show a different pattern, with relatively low PROC despite strong COMP. This domain often involves research planning, rebuttal preparation, literature synthesis, and other challenging tasks with less standardized workflows. Law Trainee and Financier have the lowest average COMP. Their workflows often require risk-oriented legal or financial judgment, which makes task completion more difficult. These results show that π-BENCH captures variation across domains. Concrete case studies are provided in App. J.

4.3 Analysis

Performance by task type. Fig. 3 (a, b, c) compares COMP and PROC on three representative task categories. Detailed per-category results are reported in App. F.3 and Tab. 8. Legal matter operations and handoffs (H) show the largest gap, with high average COMP but low average PROC (84.1% vs. 38.1%). In these cases, agents can often draft the requested document, but they fail to ensure that the matter is ready for handoff, leaving hidden intents about missing materials, blockers, and follow-up actions for the user to surface. Similarly, consumer selection, commerce, and media actions (Q) are more completion-oriented (70.8% COMP vs. 58.2% PROC). Drug design, formulation, and product benchmarking (K) show the opposite pattern, with higher PROC than COMP (84.9% vs. 68.0%). Here, hidden intents are often grounded in concrete scientific

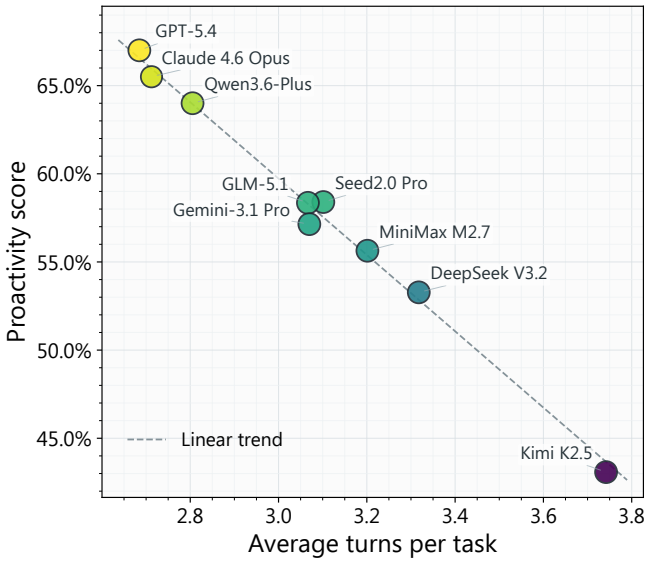

FIGURE 4 Relationship between average interaction turns per task and model-level PROC. Each point denotes one model.

constraints, such as assumptions and experimental evidence, which agents can infer more easily than they can produce a fully comprehensive technical synthesis. Overall, performance varies across task categories and metrics, suggesting that different workflow structures stress different model capabilities.

Distinguishing proactivity from completeness. Fig. 3 (d) compares aggregate PROC and COMP from Tab. **??**. The two metrics are positively related, as several high-scoring models appear near the upper-right region, but they measure different aspects of assistance. Since π-BENCH continues until each hidden intent is either resolved by the agent or supplied by the simulated user, a reactive model can still recover a reasonable COMP score after the missing requirements become explicit. PROC instead measures whether the agent reduces this burden by resolving hidden intents or eliciting them before the user has to provide them.

The off-diagonal cases make this distinction clear. Kimi K2.5 attains a relatively high COMP score but a much lower PROC score, suggesting that it can execute tasks once constraints are stated, yet often waits for the user to reveal those constraints step by step. **This contrast reveals a practical decoupling between proactivity and completeness, where reactive recovery can preserve final task quality while shifting requirement discovery back to the user.** Seed2.0 Pro shows the reverse pattern, with higher PROC than COMP, indicating that early discovery of hidden intents is not sufficient when final execution remains weak. This behavior is consistent with our evaluation design, where hidden intents capture latent interaction requirements and checklist items capture verifiable obligations in the final outcome (App. H.2). By making this gap observable, π-BENCH tests whether agents merely complete underspecified workflows after user intervention or proactively reduce the user burden while moving those workflows toward successful completion.

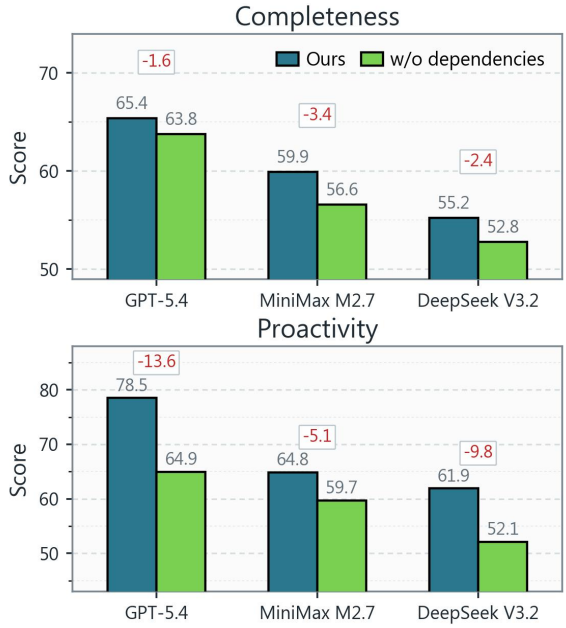

FIGURE 5 Ablation on the final task of each strong dependency group: six per user, aggregated across five roles. *Ours* uses the original trajectories, while *w/o dependencies* removes preceding sessions from the same group.

Turn count and interaction cost. Turn count provides an observable measure of the interaction cost. In π-BENCH, a session continues until each hidden intent receives a terminal status ($\mathcal{J}_{\text{com}}$, $\mathcal{J}_{\text{inf}}$, and $\mathcal{J}_{\text{pro}}$). More turns therefore indicate that the agent needed additional user input before the task became fully specified. This cost is not identical to user burden, since some extra turns may come from useful clarifications that reduce ambiguity. We therefore treat turn count as *a complementary measure rather than a substitute for* PROC. Fig. 4 shows a negative association. GPT-5.4, Claude 4.6 Opus, and Qwen3.6 Plus lie in the low-turn and high-PROC region, suggesting that they more often resolve hidden intents through early action or focused elicitation. Kimi K2.5 has the highest average turn count and the lowest PROC, which indicates that its trajectories more often depend on user-supplied information. This pattern is consistent with the results in Sec. 4.2 and supports the design goal of π-BENCH. A proactive assistant should improve final task outcomes while minimizing avoidable interaction needed to uncover and address the user's unstated requirements.

Prior interactions support proactive intent resolution. We ablate each strong dependency group to test whether earlier sessions help agents resolve later hidden intents. For each group, we remove the preceding sessions and evaluate only the final task. As shown in Fig. 5, removing history substantially reduces PROC while leaving COMP mostly stable. GPT-5.4 drops from 78.5 to 64.9, MiniMax M2.7 from 64.8 to 59.7, and DeepSeek V3.2 from 61.9 to 52.1 on PROC, with an average decrease of 9.5 points. By contrast, COMP decreases by only 2.5 points on average. This suggests that the preceding sessions are useful for resolving hidden intents before the user spells them out. Once those sessions are removed, agents can still recover some final-task quality through later user feedback, but they lose much of the ability to act proactively. This ablation confirms the importance of prior interaction for proactive intent resolution in later tasks.

More experiments. We provide additional analyses in App. F. The judge reliability study (App. F.1) shows low disagreement ($< 4\%$) under expert audits and independent frontier model audits. App. F.2 reports hidden-intent terminal status distributions, App. F.3 breaks down performance by task type, and App. F.4 summarizes failure patterns with concrete examples from App. J.

5 Conclusions

We introduced π-Bench, a benchmark for evaluating proactive personal assistant agents in long-horizon workflows, covering 100 multi-turn tasks across five domain-specific personas with hidden intents, inter-task dependencies, and cross-session continuity. By jointly evaluating proactivity and completeness, π-Bench tests whether agents can resolve underspecified requests, reuse prior context, and produce task-ready artifacts in persistent workspaces. Experiments on nine frontier models reveal clear gaps, distinguish proactivity from completeness, and show that prior interaction history helps agents resolve hidden intents while reducing avoidable user interaction.

6 Limitations

Our benchmark has several inherent limitations. The users are simulated rather than real humans, which is difficult to avoid because long horizon evaluation with live users is costly, hard to reproduce, and difficult to scale. In addition, our experiments use a single agentic scaffold adapted from Nanobot, which provides a controlled evaluation setup but may not capture the full variation introduced by alternative scaffolds, which can add substantial adaptation effort and scaffold-specific confounds.

References

[1] Anthropic. Claude code. https://claude.com/product/claude-code, 2026. AI-powered coding assistant for developers, accessed 2026-04-16.

[2] Anthropic. Introducing Claude Opus 4.6. https://www.anthropic.com/news/claude-opus-4-6, 2026. Accessed: 2026-04-22.

[3] Bytedance Seed. Seed2.0 model card: Towards intelligence frontier for real-world complexity. https://lf3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/ljhwZthlaukjlkulzlp/seed2/0214/Seed2.0%20Model%20Card.pdf, 2026. Accessed: 2026-04-22.

[4] Yuxiang Chai, Shunye Tang, Han Xiao, Rui Liu, and Hongsheng Li. Pira-bench: A transition from reactive gui agents to gui-based proactive intent recommendation agents. *arXiv preprint arXiv:2603.08013*, 2026.

[5] Maximillian Chen, Ruoxi Sun, Tomas Pfister, and Sercan Ö Arık. Learning to clarify: Multi-turn conversations with action-based contrastive self-training. *arXiv preprint arXiv:2406.00222*, 2024.

[6] Tongbo Chen, Zhengxi Lu, Zhan Xu, Guocheng Shao, Shaohan Zhao, Fei Tang, Yong Du, Kaitao Song, Yizhou Liu, Yuchen Yan, et al. Knowu-bench: Towards interactive, proactive, and personalized mobile agent evaluation. *arXiv preprint arXiv:2604.08455*, 2026.

[7] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

[8] Evolvent AI Research. Clawmark: A living-world benchmark for multi-day, multimodal coworker agents. https://evolvent.co/en/research/clawmark, 2026. Published 2026-04-13, accessed 2026-04-16.

[9] Google DeepMind. Gemini 3.1 pro. https://deepmind.google/models/gemini/pro/, 2026. Accessed: 2026-04-22.

[10] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.

[11] HKUDS. nanobot: Ultra-lightweight personal ai agent. https://github.com/HKUDS/nanobot, 2026. Open-source personal AI agent, accessed 2026-04-16.

[12] Haonian Ji, Kaiwen Xiong, Siwei Han, Peng Xia, Shi Qiu, Yiyang Zhou, Jiaqi Liu, Jinlong Li, Bingzhou Li, Zeyu Zheng, et al. Clawarena: Benchmarking ai agents in evolving information environments. *arXiv preprint arXiv:2604.04202*, 2026.

[13] Kirandeep Kaur, Vinayak Gupta, Aditya Gupta, and Chirag Shah. The proper approach to proactivity: Benchmarking and advancing knowledge gap navigation. *arXiv preprint arXiv:2601.09926*, 2026.

[14] Serin Kim, Sangam Lee, and Dongha Lee. Persona2web: Benchmarking personalized web agents for contextual reasoning with user history. *arXiv preprint arXiv:2602.17003*, 2026.

[15] Dezhi Kong, Zhengzhao Feng, Qiliang Liang, Hao Wang, Haofei Sun, Changpeng Yang, Yang Li, Peng Zhou, Shuai Nie, Hongzhen Wang, et al. Proactivemobile: A comprehensive benchmark for boosting proactive intelligence on mobile devices. *arXiv preprint arXiv:2602.21858*, 2026.

[16] Xiangyi Li, Kyoung Whan Choe, Yimin Liu, Xiaokun Chen, Chujun Tao, Bingran You, Wenbo Chen, Zonglin Di, Jiankai Sun, Shenghan Zheng, et al. Clawsbench: Evaluating capability and safety of llm productivity agents in simulated workspaces. *arXiv preprint arXiv:2604.05172*, 2026.

[17] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[18] Guangyi Liu, Pengxiang Zhao, Yaozhen Liang, Qinyi Luo, Shunye Tang, Yuxiang Chai, Weifeng Lin, Han Xiao, WenHao Wang, Siheng Chen, et al. Memgui-bench: Benchmarking memory of mobile gui agents in dynamic environments. *arXiv preprint arXiv:2602.06075*, 2026.

[19] Shuochen Liu, Junyi Zhu, Long Shu, Junda Lin, Yuhao Chen, Haotian Zhang, Chao Zhang, Derong Xu, Jia Li, Bo Tang, et al. Perma: Benchmarking personalized memory agents via event-driven preference and realistic task environments. *arXiv preprint arXiv:2603.23231*, 2026.

[20] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.

[21] Yibo Lyu, Gongwei Chen, Rui Shao, Weili Guan, and Liqiang Nie. Personalalign: Hierarchical implicit intent alignment for personalized gui agent with long-term user-centric records. *arXiv preprint arXiv:2601.09636*, 2026.

[22] Shiva Krishna Reddy Malay, Shravan Nayak, Jishnu Sethumadhavan Nair, Sagar Davasam, Aman Tiwari, Sathwik Tejaswi Madhusudhan, Sridhar Krishna Nemala, Srinivas Sunkara, and Sai Rajeswar. Enterpriseops-gym: Environments and evaluations for stateful agentic planning and tool use in enterprise settings. *arXiv preprint arXiv:2603.13594*, 2026.

[23] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.

[24] MiniMax. Minimax m2.7: Early echoes of self-evolution. https://www.minimax.io/news/minimax-m27-en, 2026. Accessed: 2026-04-22.

[25] Zairah Mustahsan, Abel Lim, Megna Anand, Saahil Jain, and Bryan McCann. Stochasticity in agentic evaluations: Quantifying inconsistency with intraclass correlation. *arXiv preprint arXiv:2512.06710*, 2025.

[26] Deepak Nathani, Cheng Zhang, Chang Huan, Jiaming Shan, Yinfei Yang, Alkesh Patel, Zhe Gan, William Yang Wang, Michael Saxon, and Xin Eric Wang. Proactive agent research environment: Simulating active users to evaluate proactive assistants. *arXiv preprint arXiv:2604.00842*, 2026.

[27] Hongyi Nie, Xunyuan Liu, Yudong Bai, Yaqing Wang, Yang Liu, Quanming Yao, and Zhen Wang. Pspa-bench: A personalized benchmark for smartphone gui agent. *arXiv preprint arXiv:2603.29318*, 2026.

[28] OpenAI. Introducing GPT-5.4. https://openai.com/index/introducing-gpt-5-4, 2026. Accessed: 2026-04-22.

[29] OpenClaw. Openclaw. https://github.com/openclaw/openclaw, 2026. Open-source personal AI assistant, accessed 2026-04-16.

[30] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.

[31] Qwen Team. Qwen3.6-Plus: Towards real world agents, April 2026. URL https://qwen.ai/blog?id=qwen3.6.

[32] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[33] Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2actbench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. *arXiv preprint arXiv:2601.19935*, 2026.

[34] Jiazheng Sun, Mingxuan Li, Yingying Zhang, Jiayang Niu, Yachen Wu, Ruihan Jin, Shuyu Lei, Pengrongrui Tan, Zongyu Zhang, Ruoyi Wang, et al. Ambibench: Benchmarking mobile gui agents beyond one-shot instructions in the wild. *arXiv preprint arXiv:2602.11750*, 2026.

[35] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

[36] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

[37] Bertie Vidgen, Austin Mann, Abby Fennelly, John Wright Stanly, Lucas Rothman, Marco Burstein, Julien Benchek, David Ostrofsky, Anirudh Ravichandran, Debnil Sur, et al. Apex-agents. *arXiv preprint arXiv:2601.14242*, 2026.

[38] Sanidhya Vijayvargiya, Vijay Viswanathan, and Graham Neubig. Asking what matters: Reward-driven clarification for software engineering tasks. *arXiv preprint arXiv:2604.14624*, 2026.

[39] Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, et al. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026.

[40] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

[41] Zhifei Xie, Zongzheng Hu, Fangda Ye, Xin Zhang, Haobo Chai, Zihang Liu, Pengcheng Wu, Guibin Zhang, Yue Liao, Xiaobin Hu, et al. Pask: Toward intent-aware proactive agents with long-term memory. *arXiv preprint arXiv:2604.08000*, 2026.

[42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[43] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[44] Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026.

[45] Z.ai. Glm-5.1: Towards long-horizon tasks. https://z.ai/blog/glm-5.1, 2026. Accessed: 2026-04-22.

[46] Weizhi Zhang, Xiaokai Wei, Wei-Chieh Huang, Zheng Hui, Chen Wang, Michelle Gong, and Philip S Yu. Memorycd: Benchmarking long-context user memory of llm agents for lifelong cross-domain personalization. *arXiv preprint arXiv:2603.25973*, 2026.

[47] Yuxuan Zhang, Yubo Wang, Yipeng Zhu, Penghui Du, Junwen Miao, Xuan Lu, Wendong Xu, Yunzhuo Hao, Songcheng Cai, Xiaochen Wang, et al. Clawbench: Can ai agents complete everyday online tasks? *arXiv preprint arXiv:2604.08523*, 2026.

[48] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

Appendix

A Benchmark Statistics

A.1 Task and Grader Statistics

In this subsection, we summarize the task and grader statistics for the 100 tasks across all users, as shown in Tab. 2.

User	Task	Session	Hidden Intent	Rubric	Rule
Researcher	20	20	84	125	31
Marketer	20	20	74	58	11
Law Trainee	20	20	89	131	43
Pharmacist	20	20	195	85	25
Financier	20	20	82	111	58
Total	**100**	**100**	**524**	**510**	**168**

TABLE 2 Task and Grader Statistics.

A.2 Tool and Skill Statistics

Tabs. 3 and 4 summarize the tool and skill inventory of the benchmark. In total, the benchmark contains 187 unique tools and 21 skills.

TABLE 3 Benchmark tools grouped by application domain.

Domain	Tool	Function
Amazon	Add Address	Add a saved address in Amazon.
	Add Product to Cart	Add a product to the cart in Amazon.
	Add Product to Wish List	Add a product to the wish list in Amazon.
	Add to History	Add a product to browsing history in Amazon.
	Clear Cart	Remove all items from the shopping cart in Amazon.
	Log In	Authenticate in Amazon.
	Log Out	Sign out of Amazon.
	Move Cart Item to Wish List	Move a cart item to the wish list in Amazon.
	Move Wish List Item to Cart	Move a wish-list item to the cart in Amazon.
	Place Order	Complete a purchase in Amazon.

Domain	Tool	Function
	Post Product Question	Post a product question in Amazon.
	Search Product Types	Search product categories in Amazon.
	Search Products	Search products in Amazon.
	Search Sellers	Search sellers in Amazon.
	Update Cart Quantity	Change product quantity in the cart in Amazon.
	View Account	View account information in Amazon.
	View Addresses	View saved addresses in Amazon.
	View Cart	View the shopping cart in Amazon.
	View Last Purchase	View the most recent product purchase in Amazon.
	View Order	View an order in Amazon.
	View Orders	View orders in Amazon.
	View Payment Card	View a stored payment card in Amazon.
	View Payment Cards	View stored payment cards in Amazon.
	View Product	View product in Amazon.
	View Product Options	View available options for a product in Amazon.
	View Product Purchases	View past purchases of a product in Amazon.
	View Product Q&A	View product questions and answers in Amazon.
	View Product Questions	View product questions in Amazon.
	View Product Reviews	View product reviews in Amazon.
	View Recommendations	View recommended products in Amazon.
	View Seller	View seller information in Amazon.
	View Wish List	View the wish list in Amazon.
	Write Product Review	Write a product review in Amazon.
Gmail	Archive Thread	Archive an email thread in Gmail.
	Create Draft	Create an email draft in Gmail.
	Forward Thread	Forward an email thread in Gmail.
	Label Thread	Apply a label to an email thread in Gmail.
	Log In	Authenticate in Gmail.
	Log Out	Sign out of Gmail.
	Mark Read	Mark an email thread as read in Gmail.
	Mark Unread	Mark an email thread as unread in Gmail.
	Remove Label	Remove a label from an email thread in Gmail.
	Reply to Email	Reply to an email in Gmail.
	Search Labels	Search labels in Gmail.
	Search Users	Search users in Gmail.
	Send Draft	Send a previously saved draft in Gmail.
	Send Email	Send an email in Gmail.
	Sign Up	Create an account in Gmail.
	Star Thread	Star an email thread in Gmail.
	Update Draft	Edit an email draft in Gmail.
	View Category Sizes	View category sizes in Gmail.
	View Draft	View draft in Gmail.

Domain	Tool	Function
	View Drafts	View drafts in Gmail.
	View Email	View email in Gmail.
	View Inbox Threads	View inbox threads in Gmail.
	View Outbox Threads	View outbox threads in Gmail.
	View Starred Threads	View starred threads in Gmail.
	View Thread	View thread in Gmail.
Phone	Add Contact	Add contact in Phone.
	Create Alarm	Create an alarm in Phone.
	Get Date and Time	Retrieve the current date and time from Phone.
	Log In	Authenticate in Phone.
	Search Contacts	Search contacts in Phone.
	Search Messages	Search text messages in Phone.
	Send Text Message	Send a text message in Phone.
	Update Alarm	Update an alarm in Phone.
	Update Contact	Update contact in Phone.
	View Alarm	View an alarm in Phone.
	View Message Thread	View a text-message thread in Phone.
	View Text Message	View a text message in Phone.
Simple Note	Add Content to Note	Add content to note in Simple Note.
	Create Note	Create note in Simple Note.
	Delete Note	Delete note in Simple Note.
	Log In	Authenticate in Simple Note.
	Log Out	Sign out of Simple Note.
	Search Notes	Search notes in Simple Note.
	Update Note	Update note in Simple Note.
	View Account	View account information in Simple Note.
	View Note	View note in Simple Note.
	View Profile	View profile in Simple Note.
Splitwise	Add Group Member	Add a member to a group in Splitwise.
	Create Group	Create group in Splitwise.
	Delete Expense	Delete expense in Splitwise.
	Log In	Authenticate in Splitwise.
	Record Expense	Record an expense in Splitwise.
	Search Users	Search users in Splitwise.
	Update Expense	Update expense in Splitwise.
	View Group	View group in Splitwise.
	View Group Balance	View the balance of a group in Splitwise.
	View Group Expenses	View expenses recorded in a group in Splitwise.
	View Groups	View groups in Splitwise.
	View Member Balances	View per-member balances in Splitwise.
	View Person Balance	View the balance of one person in Splitwise.

Domain	Tool	Function
Spotify	Add Song to Playlist	Add a song to a playlist in Spotify.
	Add to Queue	Add an item to the playback queue in Spotify.
	Create Playlist	Create playlist in Spotify.
	Follow Artist	Follow an artist in Spotify.
	Log In	Authenticate in Spotify.
	Log Out	Sign out of Spotify.
	Play Music	Start music playback in Spotify.
	Previous Song	Return to the previous song in Spotify.
	Remove Downloaded Song	Remove a downloaded song in Spotify.
	Remove Song from Playlist	Remove a song from a playlist in Spotify.
	Remove Song from Queue	Remove a song from the playback queue in Spotify.
	Review Song	Write a song review in Spotify.
	Save Album	Save an album in Spotify.
	Save Album to Library	Save an album to the library in Spotify.
	Save Playlist	Save a playlist in Spotify.
	Save Song	Save a song in Spotify.
	Save Song to Library	Save a song to the library in Spotify.
	Search Albums	Search albums in Spotify.
	Search Artists	Search artists in Spotify.
	Search Playlists	Search playlists in Spotify.
	Search Songs	Search songs in Spotify.
	Search Users	Search users in Spotify.
	Update Song Review	Edit a song review in Spotify.
	View Album	View album in Spotify.
	View Album Library	View album library in Spotify.
	View Artist	View artist in Spotify.
	View Current Song	View the currently playing song in Spotify.
	View Followed Artists	View artist following in Spotify.
	View Genres	View genres in Spotify.
	View Liked Songs	View liked songs in Spotify.
	View Playlist	View playlist in Spotify.
	View Playlist Library	View playlist library in Spotify.
	View Queue	View the playback queue in Spotify.
	View Song	View song in Spotify.
	View Song Library	View song library in Spotify.
	View Song Review	View a song review in Spotify.
	View Song Reviews	View song reviews in Spotify.
Todoist	Add Task Label	Add a label to a task in Todoist.
	Assign Task	Assign or unassign a task in Todoist.
	Create Label	Create label in Todoist.
	Create Project	Create project in Todoist.
	Create Section	Create section in Todoist.

Domain	Tool	Function
	Create Subtask	Create a subtask in Todoist.
	Create Task	Create task in Todoist.
	Delete Label	Delete label in Todoist.
	Delete Notifications	Delete notifications in Todoist.
	Delete Project	Delete project in Todoist.
	Delete Section	Delete section in Todoist.
	Delete Subtask	Delete a subtask in Todoist.
	Delete Task	Delete task in Todoist.
	Delete Task Comment	Delete a task comment in Todoist.
	Log In	Authenticate in Todoist.
	Log Out	Sign out of Todoist.
	Mark Notification	Mark a notification in Todoist.
	Post Task Comment	Post a comment on a task in Todoist.
	Remove Task Label	Remove a label from a task in Todoist.
	Search Labels	Search labels in Todoist.
	Search Users	Search users in Todoist.
	Update Project	Update project in Todoist.
	Update Section	Update section in Todoist.
	Update Subtask	Update a subtask in Todoist.
	Update Task	Update task in Todoist.
	Update Task Comment	Update a task comment in Todoist.
	View Notification Count	View the notification count in Todoist.
	View Notifications	View notifications in Todoist.
	View Project	View project in Todoist.
	View Projects	View projects in Todoist.
	View Sections	View sections in Todoist.
	View Subtasks	View subtasks in Todoist.
	View Task	View task in Todoist.
	View Task Comment	View a task comment in Todoist.
	View Task Comments	View task comments in Todoist.
	View Tasks	View tasks in Todoist.
File System	Copy Directory	Copy a directory in the File System.
	Copy File	Copy a file in the File System.
	Create Directory	Create a directory in the File System.
	Create File	Create a file in the File System.
	Log In	Authenticate in File System.
	Move Directory	Move a directory in the File System.
	Move File	Move a file in the File System.
General	Ask User	Pause execution and ask the user for required input.
	Read File	Read text, image, or document content from a file.
	Write File	Write or overwrite file content.
	Edit File	Make targeted edits by replacing matched text in a file.

Domain	Tool	Function
	List Directory	List directory contents, optionally recursively.
	Glob Search	Find files by glob pattern.
	Grep Search	Search file contents with pattern matching.
	Edit Notebook	Edit cells in a Jupyter notebook.
	Shell Exec	Run a shell command and return its output.
	Send Message	Send a message or file attachment back to the user.
	Spawn Subagent	Launch a background subagent for an independent task.
	Schedule Task	Add, list, or remove scheduled reminders and recurring tasks.
Web	Web Search	Search the web and return titles, URLs, and snippets.
	Web Fetch	Fetch a URL and extract readable page content.

TABLE 4 Skill information of the benchmark.

Skill	Function
Jargon Translator	Converts plain language into workplace jargon and translates jargon back into plain speech.
PubMed Search	Retrieves biomedical literature in a PubMed-style workflow for pharmacy and oncology tasks.
Local File Grounding	Grounds responses in local task files such as TXT, CSV, JSON, SVG, and Markdown artifacts.
PDF Reader	Extracts and grounds answers in local PDF content, including figures, legends, and methods.
JSON Translator	Translates text fields inside JSON files, especially description-heavy structured data.
Customer Requirement Analysis	Analyzes investor communication materials and turns them into standardized advisory-needs reports.
iFinD Finance Data Search	Retrieves market, fund, futures, and macroeconomic data through natural-language finance queries.
China Tax Law	Assists with Chinese tax-law research, compliance checks, planning, and dispute analysis.
Python Data Analysis	Provides lightweight Python-based data cleaning, statistical analysis, and visualization guidance.
Data Analysis	Turns raw data into reports, visualizations, and decision-oriented summaries.
Planning	Turns goals and constraints into sequenced execution plans with priorities and time blocks.
News Aggregator	Collects and summarizes domestic and international news across multiple topics.
Defense Lawyer	Supports criminal-defense analysis, evidence assessment, strategy design, and drafting.
Corporate Lawyer	Supports transaction-oriented legal work such as contract review, compliance checks, and risk assessment.

Skill	Function
Proactive Task Validator	Validates whether hidden intents, dependencies, and checklists are aligned in proactive-task datasets.
One-Page Credit Analysis	Structures a one-page credit view for Chinese-market financial analysis and risk review.
Financial Analysis	Supports portfolio analysis, risk attribution, and backtesting-oriented financial reporting.
Web Browsing	Checks authoritative public webpages when local context alone is insufficient.
Local Web Search	Runs targeted web search for product, formulation, and market-comparison tasks.
Medical Literature Reader Pro	Reads, compares, and synthesizes biomedical papers and evidence chains.
Law Exam Trainer	Builds and explains law-exam practice materials from videos or documents.

A.3 Taxonomy of Benchmark Tasks

In this subsection, we detail the categorization of the 100 benchmark tasks designed for our evaluation. To systematically assess the multimodal agents across diverse professional and daily scenarios, we group the tasks into 18 fine-grained categories. Rather than relying on broad topical divisions, this classification system strictly emphasizes the specific action intents, reasoning requirements, and underlying workflows inherent to each task. A comprehensive overview of these categories, along with their corresponding descriptions and covered task lists, is presented in Tab. 5.

TABLE 5 Taxonomy of the 100 benchmark tasks across five user profiles.

Type	Task category	Description	Count
A	AI Research Frontier Intelligence	Tasks that track multimodal agents, think-with-image methods, OpenClaw updates, benchmark metrics, paper links, and follow-up value. They emphasize selective reading, reproducibility signals, and research-roadmap judgment rather than broad literature dumping.	7
B	Scholarly Experiment And Rebuttal Artifacts	Tasks that convert experiment records, peer-review comments, and paper-writing conventions into structured research deliverables. The common pattern is disciplined triage, table-first result synthesis, and reusable Markdown or LaTeX outputs for academic workflows.	4
C	Researcher Life And Career Planning	Tasks that blend personal constraints with concrete recommendations for campus life, housing, fitness, diet, and research-oriented internships. They require practical comparison tables and decisions anchored to the user's body metrics, location, budget, and academic trajectory.	4
D	Financial Model Validation And Governance	Tasks centered on independent validation conclusions, risk-priority frameworks, champion-model decisions, and regulatory-style effective challenge. They favor conclusion-first writing, model-use implications, limitations, monitoring, and remediation over formula tutorials.	6

Type	Task category	Description	Count
E	Quantitative Data Engineering And Curve Analytics	Tasks that ask for financial data sources, preprocessing logic, model calibration, normalization, and executable analytical scripts. They are grouped by the need to make data pipelines statistically defensible and source-of-truth aware.	9
F	Legal Pleading And Contract Drafting	Tasks that produce or revise formal legal instruments: complaints, defense briefs, loan agreements, and memoranda. The unifying requirement is legally conventional structure, precise prayers or arguments, and fact-to-law alignment.	7
G	Legal Compliance And Evidence Strategy	Tasks that require legal reasoning before drafting: compliance ratings, subjective-objective decomposition, similar-case factors, and evidence collection. They test whether the agent can convert legal analysis into litigation or compliance strategy.	6
H	Legal Matter Operations And Handoffs	Tasks that operationalize legal work through notes, emails, SMS messages, temporary boards, and filing reminders. They are practical legal-support workflows where timing, missing materials, blockers, and cleanup confirmation matter.	5
I	Biomedical Literature And Clinical Evidence	Tasks that read pharmacy, oncology, and clinical-study materials, often correcting flawed local drafts. They focus on literature storylines, figure interpretation, study workflow, treatment pathways, and clinically meaningful evidence synthesis.	5
J	Experimental Assay And Lab Result Reasoning	Tasks that turn raw experimental materials into interpretable analysis workflows. They are narrower than literature review tasks because the central work is assay design, Ct-value handling, controls, replicates, and result interpretation.	2
K	Drug Design, Formulation And Product Benchmarking	Tasks that reason from molecular properties, topical product landscapes, formulation routes, PROTAC fundamentals, and linker design. They share an early-stage R&D pattern: separate known facts from assumptions, screen candidate routes, and keep design choices experimentally grounded.	5
L	Laboratory Procurement And Research Logistics	Tasks that convert experiment plans and inventory gaps into purchasing priorities. They are grouped by procurement triage, supplier-aware lists, and sequencing around experiment blockers.	3
M	Market Intelligence And Brand Research	Tasks that extract market size, regional dynamics, product philosophy, risk signals, technical specifications, and brand doctrine. The emphasis is research-grounded marketing input rather than finished copy alone.	4
N	Marketing Content Systems And Conversion Copy	Tasks that create scripts, A/B headlines, course funnels, SOPs, and product asset briefs under explicit channel rules. They test whether the agent can preserve audience pain points, hooks, role boundaries, and conversion incentives across reusable content systems.	7
O	Co-Branding Strategy And Creative Governance	Tasks that synthesize brand doctrines, veto rules, visual constraints, product-form constraints, and launch mechanics into co-branding proposals. The category is defined by creative governance: the agent must respect jointly established rules while still producing a concrete campaign concept.	3
P	Crisis, Recovery And Reputation Communications	Tasks that handle incident facts, public apology language, compensation framing, and post-crisis recovery planning. They require careful recall of approved numbers and causes, plus restraint against fabricating unsupported commitments.	3

Type	Task category	Description	Count
Q	Consumer Selection, Commerce And Media Actions	Tasks that compare products or media items, choose according to latent preferences, and complete visible commerce or media actions. They include carts, wish lists, playlists, emails, and recommendation rationales where price, fit, rating, inventory, or genre constraints are decisive.	9
R	Tool-Mediated Administrative Workflows	Tasks that require closed-loop use of productivity, communication, finance-splitting, or file-system tools. The distinctive feature is not the domain content but the auditable action sequence: read local context, perform the write operation, verify the result, and sometimes clean up temporary state.	11

B User Agent Protocol

B.1 Overview

The user agent is instantiated with GPT-5.4 as the base model and simulates the user side of each session. It controls how hidden requirements are revealed during interaction. Given the current dialogue, the latest agent response, and the task-level hidden intents, it updates the intent tracking state and produces the next user message when the session should continue. This protocol separates user simulation from final task grading. Specifically, the user agent determines whether hidden intents have been proactively resolved or need to be revealed, while checklist-based graders evaluate whether the final artifacts satisfy the task requirements.

To support stable trajectory-level evaluation, we decompose the user agent protocol into two stages rather than relying on a single free-form user simulator to both judge the agent response and generate the next message. The first stage checks whether the agent has already satisfied any hidden intents through its response, tool use, or workspace updates. The second stage determines whether the agent has asked a targeted clarification question and, if needed, generates the next user message. This design makes terminal intent assignment more explicit while preserving natural multi-turn interaction.

B.2 Two-Stage Intent Assignment

At each turn, the user agent only considers hidden intents that have not yet received a terminal status. Once an intent is assigned *completed*, *inferred*, or *provided*, it is removed from subsequent tracking within the same session.

Assignment is performed in two stages. In the first stage, the user agent checks whether the latest agent response has already satisfied one or more unresolved intents. Evidence may come from the response content, a tool call, or a newly created or modified artifact. If an intent is satisfied without being explicitly stated by the user, it is assigned *completed*. This stage has priority because it captures the strongest form of proactivity, where the agent directly acts on an unstated requirement rather than asking the user to provide it.

Prompt – Completion Checking

```
## Assistant Response
<assistant_message>
{assistant_message}
</assistant_message>

## Files Read Context
Use these file contents as additional context for judging intent
satisfaction.
{files_context_xml}

## Hidden Intent Status Snapshot
Judge only the hidden intents listed here.
{status_hidden_intents_xml}

## Objective
Decide whether the assistant response, together with the files-read
context above, already reflects each listed hidden intent.

## Evaluation Policy
1. Judge from the assistant response and the files-read context above.
2. Be strict and objective. The assistant must precisely and explicitly
hit the hidden intent.
3. A hidden intent is satisfied only if the assistant provides specific,
detailed explanations or concrete actions in the response. Vague or
generic answers do not count.
4. Fully trust the assistant's wording and the files-read context, but
strictly evaluate the level of detail provided.
5. Do not call tools or check factual accuracy beyond the provided
context.
6. YES means the response and context precisely address the hidden intent
with specific details.
7. NO means the response and context do not precisely hit the intent, or
lack specific details.

## Output Format
Output only XML blocks in the following shape.
<c1>
<content>
{hidden_intent_content}
</content>
<decision>
YES or NO
</decision>
</c1>
```

In the second stage, the user agent checks whether the latest agent response contains a clarification question. If the question directly targets one or more unresolved intents, those intents are assigned *inferred*, and the user agent answers the corresponding missing requirements in the next message. A question is considered targeted only when it asks for information needed to resolve a specific hidden intent. Generic questions such as asking whether the user has any other preferences are not sufficient unless they clearly identify the missing requirement.

If no targeted clarification question is found, the user agent selects one unresolved intent that is relevant to the current stage of the task and reveals the corresponding information in the next message. The selected intent is then assigned *provided*. This case reflects a weaker interaction pattern, where the user must supply missing information because the agent neither completed the intent nor elicited it through a focused question.

Prompt – Clarification Checking

```
## Assistant Response
<assistant_message>
{assistant_message}
</assistant_message>

## Hidden Intents Still Not Provided
Judge only the hidden intents listed here.
{status_hidden_intents_xml}

## Objective
Decide whether the assistant response contains a clear follow-up question
that is specifically about each listed hidden intent.

## Evaluation Policy
1. Judge from the assistant response. Consider all follow-up questions,
requests, and action suggestions inside it, not only the last sentence.
2. YES means the assistant explicitly asks about that hidden intent, asks
a very close confirmation question about it, or proposes concrete next
steps that directly correspond to it.
3. NO means the question is missing, vague, generic, or does not clearly
target that hidden intent.
4. Generic prompts such as ``anything else?'' or ``do you want to add
more?'' must be NO.
5. A question must clearly get the point. Broad topic overlap is not
enough.

## Output Format
Output only XML blocks in the following shape.
<c1>
<content>
{hidden_intent_content}
</content>
<decision>
YES or NO
</decision>
</c1>
```

This two-stage procedure induces a priority order among terminal statuses. Direct satisfaction is assigned before targeted elicitation, and targeted elicitation is assigned before user-provided information. This order matches the proactivity score in Sec. 3.5, where both *completed* and *inferred* are treated as proactive behavior, while *provided* indicates that the user had to

surface the requirement without sufficient agent initiative.

B.3 Session Termination

A session terminates after every hidden intent in $\mathcal{I}$ has received a terminal status and the agent has produced its final response. This rule provides a concrete stopping condition for the simulated interaction and avoids requiring the user agent to make a separate subjective judgment about whether the conversation has naturally ended. Once all hidden intents have been completed, inferred, or provided, the user agent has no remaining hidden requirement to reveal, and the session has reached the intended stopping point for proactivity evaluation.

This termination rule is aligned with the metric definition. Since PROC is computed from the partition of hidden intents into $\mathcal{I}_{\text{com}}$, $\mathcal{I}_{\text{inf}}$, and $\mathcal{I}_{\text{pro}}$, the session ends only after all intents have been assigned to one of these sets. The final trajectory is then passed to checklist graders for completeness evaluation, which is independent of whether each intent was completed, inferred, or provided during interaction.

C Detailed Evaluation Protocol

C.1 Overview

We evaluate each completed trajectory along two axes, proactivity and completeness. Proactivity measures whether the agent resolves hidden intents through direct action or targeted elicitation, following the definition in Sec. 3.5. Completeness measures whether the task requirements are actually satisfied by the trajectory and produced artifacts. This separation is important because a conversation may appear natural while still failing to complete the task, and a task may eventually be completed only after the user reveals requirements that the agent did not proactively identify.

For each session, evaluation is performed over the full interaction trajectory. The trajectory includes user and agent messages, tool calls, tool results, and workspace changes when applicable. Proactivity is computed from the terminal status assignment over hidden intents. Completeness is computed from a checklist of verifiable criteria, using either LLM based rubric evaluation or rule based tool evaluation. These scores are then reported at the task level and averaged across repeated runs.

C.2 Proactivity Evaluation

Proactivity follows the intent tracking procedure described in Sec. 3.4 and App. B. Each hidden intent is assigned one terminal status from {*completed*, *inferred*, *provided*} during the interaction. We use the resulting partition of hidden intents to compute PROC as defined in Sec. 3.5. When tasks form a dependency group, the relevant hidden intents from the dependent workflow are taken into account together. We do not introduce additional post hoc judgments for proactivity in this appendix, since the score is fully determined by the user agent protocol and the terminal status assignment.

C.3 Checklist Based Completeness Evaluation

Completeness is evaluated with task-specific checklists. Each checklist item describes a concrete criterion that should be satisfied by the trajectory or final artifacts. These criteria cover required files, generated outputs, filled fields, tool outcomes, formatting constraints, and other task obligations. Unlike hidden intents, which measure whether the agent proactively resolves unstated requirements, checklist items measure whether the task is ultimately completed.

We evaluate checklist items with two complementary methods:

- **Rubric-based evaluation**. Open-ended textual criteria are evaluated by an LLM rubric evaluator. The evaluator receives a rendered trace containing the interaction history and relevant task context, then assigns each criterion a strict `YES` or `NO` judgment. We map `YES` to 1 and `NO` to 0.
- **Rule-based verification**. Structured criteria that require exact verification are evaluated by task-specific Python scripts. These scripts inspect the tool history and return binary scores for the corresponding checklist items. This is useful for cases where completion depends on exact tool outcomes, such as whether an order contains the correct product and quantity.

A task may combine both methods. LLM rubric evaluation handles criteria that require semantic judgment, while rule-based evaluation handles criteria that can be checked from structured tool evidence. The resulting binary judgments are merged into a single checklist before task-level aggregation, and the final completeness score is computed as the average over checklist items.

C.4 Tool Records

Some checklist criteria require evidence from tool calls and tool results. We therefore keep structured tool records as part of the trajectory. Each record contains the tool name, the call payload, and the returned result. The following example shows a simplified order inspection record from an Amazon task. The example is used only to illustrate the structure of tool evidence used by the evaluator.

Example – Tool Call and Response json "tool$_{n}ame$" : "$mcp_{a}ppworld_{a}mazon_{s}how_{o}rder$","$call$":"$order_{i}d$":901,"$result$":"$res$

C.5 LLM Rubric Evaluation

Text checklist criteria are evaluated by an LLM rubric evaluator. The evaluator receives a rendered trace containing the interaction history and selected task-relevant context. For each task, completeness is aggregated over the interaction, so a criterion is credited if the trajectory provides sufficient evidence that it has been satisfied during the session. When tool evidence is needed, we include only the tool calls or tool results that are relevant to the criterion being evaluated. These fields are selected during task construction by domain experts.

This selective context makes rubric evaluation more reliable and easier to audit. Full tool histories can be long, repetitive, and unrelated to a specific checklist item. Providing only relevant tool evidence keeps the evaluation prompt compact while preserving the information needed to decide whether the criterion is satisfied. The selected tool fields are appended to the

rendered trace as structured context. The LLM rubric evaluator then judges each text checklist item from this trace and returns a strict YES or NO answer.

Prompt – Checklist-based Rubric Evaluation

```
## Full Hidden Intent
These are all hidden intents for the current task.
{hidden_intents_xml}

## Interaction History
<history>
{history}
</history>

## Checklist Criteria
{criteria_list}

## Objective
You are a strict evaluator.
For each checklist criterion, decide whether the interaction history
clearly satisfies it.

## Scoring Rules
1. Use only evidence from the interaction history.
2. Score YES only when the criterion is clearly satisfied.
3. Score NO when evidence is missing, ambiguous, or contradicted.
4. Do not guess.

## Output Format
Output only XML blocks in the following shape. Keep each criterion text
exactly the same as given. Each score must be YES or NO.
<c1>
<criterion>
{criterion_text}
</criterion>
<score>
YES or NO
</score>
</c1>
```

C.6 Rule Based Tool Scoring

Rule-based tool scoring is used when a checklist item requires exact verification over structured tool calls or tool results. This is useful for criteria where natural language evidence is not

sufficient, such as whether the agent actually placed an order, submitted a form with the correct fields, or retrieved the expected record from an external system. For such criteria, we use task-specific Python scripts that inspect the tool history and return binary scores for the corresponding checklist items.

This procedure is separate from the selected tool context used in LLM rubric evaluation. The rubric evaluator sees a rendered trace with only the task-relevant context selected for semantic judgment. In contrast, rule-based scoring inspects the full tool history for the turn. This separation allows the evaluation to combine flexible semantic judgment with exact checks over structured evidence.

For example, in a shopping task, the agent may claim that it placed the correct order, but completeness should be determined from the actual order record rather than from the natural language response alone. A rule-based scorer can first recover the order identifier from a successful order placement call and then verify the product and quantity through a later order inspection call.

Example – Rule Based Evaluator Python from typing import Any

TARGET$_{P}RODUCT_{I}D = 1578TARGET_{Q}UANTITY = 2$

def $_{d}efault_{s}cores() -> dict[str, int] : return"usedamazon_{place_{o}rdersuccessfully}":0,"recoveredplacedorder_{i}d":0,"show_{o}rderco$

def score(tools$_{h}istory : list[dict[str, Any]]) -> dict[str, int] : result =_{d} efault_{s}cores()order_{i}d : int|None = None$

for item in tools$_{h}istory : if notisinstance(item, dict) : continue$

tool$_{n}ame = str(item.get("tool_{n}ame")or"").strip()call_{p}ayload = item.get("call")raw_{r}esult = item.get("result")response = raw_{r}esult.get("response")if isinstance(raw_{r}esult, dict)elseNone$

if tool$_{n}ame == "mcp_{a}ppworld_{a}mazon_{place_{o}rder"andisinstance(response,dict):if(isinstance(call_{p}ayload,dict)and"payment_{c}ard_{i}d$

if tool$_{n}ame! = "mcp_{a}ppworld_{a}mazon_{show_{o}rder"ornotisinstance(response,dict):continueif order_{i}disnotNoneandresponse.get("orde$

order$_{i}tems = response.get("order_{i}tems")if notisinstance(order_{i}tems, list) : continue$

for order$_{i}teminorder_{i}tems : if notisinstance(order_{i}tem, dict) : continueif order_{i}tem.get("product_{i}d")! = TARGET_{P}RODUCT_{I}D : continueresult["show_{o}rderconfirmstargetproduct"] = 1if order_{i}tem.get("ordered_{q}uantity" TARGET_{Q}UANTITY : result["show_{o}rderconfirmsordered_{q}uantity == 2"] = 1returnresult$

return result

This example illustrates how rule-based evaluation complements LLM rubric evaluation. The LLM evaluator can assess open-ended textual criteria from the rendered trace, while the script verifies structured facts from the tool history. Both outputs are converted into binary checklist values and aggregated under the same completeness evaluation procedure.

D Reproducibility and Runtime Settings

Execution environment. All experiments were run on a Linux server with Ubuntu 24.04.1 LTS. The machine has two CPU sockets, with 16 physical cores per socket and 64 hardware threads in total. The server has 251 GiB of RAM and 1 TB of local storage. In practice, individual task runs used substantially less than 8 GB of memory, and the full project workspace, including task assets, traces, and intermediate outputs, required less than 32 GB of storage. Benchmark execution is containerized with Docker to keep the runtime environment consistent across models

and users.

Agent scaffold and app environments. All evaluated models are run under the same agentic scaffold, adapted from Nanobot [11][†] (MIT License). For tasks that require app-backed tools, we construct simulated app environments based on AppWorld [36][‡] (Apache-2.0 License). This setup keeps the interaction protocol, workspace access, and tool interface consistent across models while keeping the benchmark focus on proactive intent resolution rather than infrastructure differences.

Model access. The evaluated models are accessed through hosted model APIs and are all run under the same agent scaffold described above. Model-specific differences are therefore limited to the provider-side model behavior rather than local runtime configuration. API credentials and provider endpoints are configured outside the manuscript and are not embedded in the benchmark artifacts.

E Societal Impacts

π-Bench is intended to support safer and more reliable evaluation of proactive personal assistant agents before such systems are deployed in real workflows. By measuring whether agents can identify underspecified needs, ask targeted questions, and complete artifact-grounded tasks, the benchmark may help developers diagnose failures that would otherwise increase user effort or lead to incomplete outcomes. At the same time, proactive assistants raise important risks: an agent that infers too much may act beyond the user's intent, expose or misuse private context, or make inappropriate decisions in sensitive domains. We therefore view π-Bench as an evaluation resource rather than a deployment recommendation, and emphasize that real-world proactive agents should be paired with user control, privacy safeguards, transparent logging, and domain-specific review when tasks involve high-stakes personal, legal, medical, or financial information.

F Experiments

F.1 Reliability of Judgment Based Evaluation

Our main evaluation uses GPT-5.4 for two judgment based components. The first is rubric based checklist grading, which contributes to Comp. The second is terminal status assignment for hidden intents, which determines Proc. Since these two signals are central to π-Bench, we audit whether the judgments are stable across human and model based reviewers.

We sample 120 task trajectories uniformly across evaluated models. For each trajectory, auditors inspect all checklist judgments and all hidden intent status assignments. The human audit uses three expert annotators and reports disagreement against the original evaluation after majority aggregation. The model audit uses Claude Opus 4.6, GPT-5.4, and Gemini 3.1

[†] https://github.com/HKUDS/nanobot
[‡] https://github.com/stonybrooknlp/appworld

Pro. The GPT-5.4 audit is included as a separate judging pass to measure self consistency, while the other model audits test agreement with independent frontier judges. Tab. 6 reports the disagreement rate between the scoring run and each audit source.

Audit source	Checklist	Hidden intents
Human experts	2.66	1.48
Claude Opus 4.6	1.95	0.90
GPT-5.4	2.28	1.77
Gemini 3.1 Pro	3.51	2.05

TABLE 6 Disagreement rates for judgment based evaluation. We audit checklist grading for Comp and hidden intent status assignment for Proc. All values are percentages. Lower is better.

The audits show strong agreement with the scoring run on both axes. Human experts disagree with 2.66% of checklist judgments and 1.48% of hidden intent assignments, and the frontier model audits show a similar pattern. Disagreement stays below 3.6% for checklists and below 2.1% for hidden intents. This consistency suggests that the judgment based components of our evaluation are reliable and that the reported Proc and Comp scores are unlikely to be driven by evaluator noise. Hidden intent status is slightly more stable, likely because it follows explicit terminal categories in the user agent protocol. Checklist grading leaves more room for variation because it may require reading longer artifacts and matching them against task specific evidence. Overall, the audit supports the intended separation in π-Bench. Proc captures whether agents resolve unstated user needs, while Comp captures whether they complete the requested workflow.

F.2 Terminal Status of Hidden Intents

Table 7 provides an intent level diagnostic of how hidden intents are resolved at the end of an interaction. Completed intents are satisfied without explicit user disclosure, inferred intents are elicited through targeted agent questions, and provided intents are supplied by the simulated user without such questions. *This distribution is different from the reported proactivity score, which is computed within each task trajectory and then averaged across tasks and repeated runs.*

The main variation comes from direct completion rather than targeted elicitation. Qwen3.6 Plus has the highest completed rate at 63.18, followed by Claude Opus 4.6 at 60.56 and GPT-5.4 at 56.84. These models also have the lowest provided rates, at 26.37, 28.20, and 30.87. In contrast, Gemini 3.1 Pro, Kimi K2.5, and MiniMax M2.7 leave more hidden intents to be supplied by the user, with provided rates of 47.39, 45.85, and 43.81. Inferred rates remain low and tightly clustered across models, ranging from 8.36 to 12.29, which suggests that current models more often either resolve hidden requirements directly or wait for user disclosure rather than eliciting them through focused questions.

Model	Completed	Inferred	Provided
GPT-5.4	56.84	12.29	30.87
Gemini 3.1 Pro	44.25	8.36	47.39
Claude Opus 4.6	60.56	11.24	28.20
DeepSeek V3.2	54.58	9.35	36.07
MiniMax M2.7	45.42	10.77	43.81
Kimi K2.5	44.28	9.87	45.85
Seed2.0 Pro	50.24	11.30	38.46
GLM-5.1	56.50	8.86	34.64
Qwen3.6 Plus	63.18	10.45	26.37

TABLE 7 Distribution of terminal hidden intent statuses.

F.3 Task-Type Breakdown of Performance

Tab. 8 provides a task-type diagnostic of how Comp and Proc separate across workflow structures. Research-centered workflows (A–C) and legal matter operations and handoffs (H) show relatively high Comp but lower Proc, suggesting that agents can often produce the final deliverable once requirements are explicit, while remaining weaker at identifying latent goals, missing materials, and operational blockers earlier in the workflow. Drug design, formulation, and product benchmarking (K) shows the opposite pattern, with higher Proc than Comp; here, hidden intents are often grounded in concrete scientific constraints such as assumptions, candidate routes, and experimental evidence. Consumer selection, commerce, and media actions (Q) are more completion-oriented, but their lower Proc indicates that visible task completion does not fully capture recovery of latent preferences. Tool-mediated administrative workflows (R) are more balanced, yet still show that the two metrics capture distinct aspects of performance.

Model-level trends further support this separation. GPT-5.4 attains the strongest average Proc and performs well on legal drafting (F), crisis communication (P), and administrative workflows (R), while Claude Opus 4.6 achieves the strongest average Comp and remains competitive on research-oriented completion (A–C). Qwen3.6 Plus shows comparatively stable performance across both metrics. Overall, these results indicate that workflow structure shapes which capabilities are stressed, and that Comp and Proc provide complementary views of model behavior.

F.4 Failure Analysis

This section summarizes common failure patterns where agents either fail to satisfy checklist requirements or fail to proactively resolve hidden intents. These two signals capture different aspects of failure: checklist items measure task completion, while hidden intents measure whether requirements are resolved proactively rather than supplied by the user. We focus on broad,

Type	GPT	Gemini	Claude	DeepSeek	MiniMax	Kimi	Seed	GLM	Qwen
A	40 / 57	40 / 38	38 / **64**	24 / 52	35 / 50	25 / 37	38 / 48	**49** / 42	20 / 55
B	55 / 69	42 / 53	**62** / 73	20 / 71	30 / 67	31 / **80**	28 / 39	30 / 70	46 / 71
C	30 / 69	**46** / 86	42 / **92**	23 / 86	30 / 86	20 / 83	**46** / 88	38 / 67	40 / 83
D	75 / **61**	67 / 35	65 / 39	59 / 39	57 / 53	47 / 45	56 / 35	75 / 42	**78** / 46
E	**76** / 63	49 / 56	75 / **78**	52 / 53	49 / 63	36 / 54	57 / 52	60 / 67	74 / 59
F	**67** / 44	61 / **46**	60 / 40	36 / 26	44 / 31	40 / 37	51 / 25	56 / 41	56 / 38
G	**61** / 76	**61** / 43	34 / 63	22 / 67	18 / 68	36 / 69	39 / 41	43 / 59	41 / **78**
H	39 / 85	31 / 88	40 / 87	36 / 79	**44** / 78	36 / 82	41 / 80	36 / **89**	40 / **89**
I	83 / 65	80 / 61	**90** / 65	80 / 50	84 / 59	73 / **78**	77 / 67	84 / 73	83 / 73
J	69 / **56**	61 / 44	78 / 33	72 / 52	**80** / 41	61 / 48	**80** / 33	67 / 52	76 / 37
K	78 / 74	83 / **81**	**93** / 65	83 / 62	85 / 69	82 / 67	87 / 64	84 / 63	89 / 67
L	72 / 81	68 / 75	77 / **83**	**81** / 72	77 / 75	67 / **83**	**81** / **83**	68 / 81	79 / 72
M	**75** / 93	63 / 93	**75** / **100**	50 / **100**	42 / 93	42 / **100**	67 / **100**	67 / 93	42 / 93
N	78 / 65	69 / 55	80 / 71	75 / 53	78 / 60	50 / 70	76 / 36	64 / **75**	**81** / 65
O	**90** / 44	69 / 67	88 / 75	80 / 61	85 / 67	54 / **78**	82 / 56	85 / 72	82 / 44
P	**79** / 81	49 / 75	54 / **89**	62 / 69	46 / 70	23 / 29	52 / 40	34 / 51	62 / 76
Q	61 / 75	50 / 69	67 / 74	56 / 69	58 / 61	35 / 66	65 / 66	62 / 77	**70** / **80**
R	**72** / 68	58 / **72**	69 / 70	64 / 62	57 / 64	45 / 64	55 / 60	60 / 70	71 / 65

TABLE 8 Performance by task workflow type. Each cell reports PROC / COMP in percentages. Bold and underline mark the best and second-best scores within each row and metric, including ties. Model names are abbreviated and full names are given in Sec. 4.1. Type labels follow Tab. 5.

recurring behaviors rather than task-specific corner cases, and provide concrete examples in the case studies in App. J.

Ignoring recoverable prior context. Some agents treat the current user message as a standalone request even when the task depends on information established in earlier sessions. This leads to missed hidden intents or incorrect final artifacts, as shown in the Researcher dependency failure in Fig. 21 and the meal-planning contrast in Fig. 9.

Completing the visible request while missing hidden requirements. Agents often produce a plausible answer for the explicit request but leave implicit preferences, formatting constraints, or task-specific requirements unresolved until the user states them. Fig. 7 illustrates this pattern through several hidden intents marked as user-provided rather than agent-completed, and Fig. 11 shows that even a response with high checklist completeness can still rely on user-provided hidden requirements.

Failing to ask targeted clarifications. When information is genuinely missing, clarification is useful only if it targets the specific requirement needed to complete the task. The targeted-followup successes in Fig. 13 and Fig. 15 serve as positive contrasts: weaker trajectories instead rely on repeated user intervention before hidden requirements are surfaced.

Using tools without verifying the required artifact. Some failures occur after the agent invokes relevant tools but does not verify that the produced artifact contains the required content or that the final state matches the checklist. The Law Trainee comparison in Fig. 18 shows this gap: the agent uses SMS and Todoist tools, but the final trace does not preserve the required handover details or reminder checks.

G Task Format

Each task is stored as a structured configuration that specifies the user-facing request, latent requirements, evaluation objectives, and metadata. The example below shows a survey-paper task, where earlier tasks in the same group may give the agent opportunities to infer the hidden intents needed for this final task. Task-specific names, links, and sensitive details are anonymized.

Example Task Formattcbyaml title: "Survey Recent Papers on a User-Specified Research Topic"

description: > The user is starting a small research project and asks the agent to organize a set of relevant papers from a local paper list. The task requires the agent to infer the user's preferred topic focus, filter candidate papers, and prepare a concise reading plan that supports later topic selection.

task$_{t}ype$: $long_{t}erm$

trigger: type: user

intent: initial$_{i}nput$:> $Iwrotealocalfileat[FILE_{N}AME]thatcontainsrecentlyacceptedpapers.Pleaseorganizesevera$

hidden$_{i}ntent$: $-content$: $"Prioritizepapersrelatedto[RESEARCH_{T}HEME]."$ $-content$: $"Recommendaround[N]papersratherthanproducingalongsurvey."$ $-content$:> $Foreachpaper,provideabriefintroductionandsummarizethekeytechnicalpoints.-content$:> $Includetheofficialpaperlinkwhenavailable.-content$:> $Includetheprojectorcodelinkwhenavailable;otherwiseexp$ $content$:> $Foreachpaper,statewhetheritissuitabletofollowandexplainhowtheusershouldfollowit.$

objectives: checklist: - criterion: "Include [PAPER$_{T}ITLE_{1}$]."$-criterion$:> $Explainthat[PAPER_{T}ITLE_{1}]studies[CO$ $criterion$: $"Writetheofficialpaperlinkfor[PAPER_{T}ITLE_{1}]as[PAPER_{L}INK_{1}]."$ $-criterion$: $"Writethecodelinkfor[PAPER_{T}ITLE_{1}]as[CODE_{L}INK_{1}]."$

- .....

- criterion: > For every recommended paper, state whether it is suitable to follow. - criterion: > For every recommended paper, give a concrete follow-up action, such as reading the method section, reproducing the code, or comparing the task formulation with the user's project.

metadata: difficulty: hard

H Discussion

H.1 Metric Relationship

Proactivity and completeness are related but intentionally separated. In real user-agent interaction, a final outcome may improve either because the agent proactively identifies hidden requirements or because the user eventually spells them out. π-Bench follows the latter interac-

tion to exhaustion. The simulated user continues until every hidden intent has been completed, elicited, or explicitly provided. Thus, the final trajectory contains the complete set of user requirements, either through the agent's own initiative or through oracle-like user provision. Under this protocol, PROC measures who drives requirement discovery, while COMP measures whether the agent converts the resulting trajectory into correct artifacts and decisions. The two scores therefore need not move together. A reactive agent can obtain high COMP after receiving many user-provided intents, while a proactive agent can still lose COMP by mishandling files, tools, formats, or downstream constraints. This separation lets us analyze the trade-off between surfacing latent requirements and completing the final workflow.

H.2 Hidden Intents and Checklists

Hidden intents and checklists serve different roles in our benchmark.

Hidden intents. Hidden intents describe latent requirements that are not stated in the initial request but should guide the agent's behavior during interaction. They capture what a proactive assistant should infer, ask about, or act on before the user explicitly provides the information. Such intents may involve user preferences, task constraints, output conventions, or cross-session dependencies. Therefore, hidden intents are used to evaluate how the agent handles underspecification and whether it reduces the need for user intervention.

Checklists. Checklists, in contrast, define verifiable criteria for task completion. They specify what must be true of the final trajectory or produced artifacts, such as whether a required file is created, whether an output follows the requested format, whether a tool action succeeds, or whether a generated artifact contains the necessary content. A checklist item may depend on a hidden intent, but it is evaluated only as an outcome requirement. This separation lets us distinguish proactive behavior from final correctness. An agent may complete a task after the user provides all missing requirements, leading to high completeness but lower proactivity. Conversely, an agent may correctly identify hidden intents but still fail to satisfy some concrete checklist items, leading to high proactivity but lower completeness.

I Benchmark Construction and Annotation

I.1 Task Sources and Workflow Collection

Workflow sources. Tasks are derived from realistic workflows associated with the five user roles in π-BENCH: researcher, marketer, law trainee, pharmacist, and financier. For each role, we first collect representative work routines, common deliverables, and supporting materials from domain experts and public task patterns. These sources cover both professional work and everyday knowledge work, such as experiment analysis, paper writing, content planning, document review, literature organization, financial reconciliation, and report generation.

Selection criteria. During collection, we focus on workflows that are artifact-centric and naturally require interaction. We also require the workflow to contain realistic underspecification.

In other words, the task should not be fully determined by the first user message. Instead, correct completion should depend on constraints, preferences, prior decisions, or workspace context that the agent needs to recover through proactive behavior.

Data sanitization. We do not directly use private or sensitive real-world data. When a workflow is inspired by actual work practice, annotators rewrite and normalize the materials into synthetic but realistic task instances. This includes replacing private names, removing sensitive details, simplifying irrelevant background, and ensuring that all required information can be accessed through the provided workspace, memory, or interaction protocol.

I.2 Task Construction Procedure

Task goal selection. For each candidate workflow, annotators first define a task goal that is realistic, checkable, and well-scoped. The goal should reflect a plausible need of the corresponding user role, cover a meaningful part of the workflow, and admit clear evidence for whether the agent has completed it. We avoid goals that are either too narrow to require interaction or too broad to support reliable evaluation.

Task specification. Annotators then convert the workflow into a concrete task specification. This process includes:

- Decide what information should appear in the initial request and what information should remain implicit as hidden intents.
- Prepare the workspace state and supporting materials, such as input files, prior artifacts, structured records, or tool-accessible information.
- Specify cross-session dependencies when earlier decisions, files, or user preferences should influence the current session.

The initial request must be underspecified enough to test proactivity, while still being concrete enough for the agent to begin useful work.

Intent and checklist annotation. Annotators next define the hidden intents and checklist items. Hidden intents capture latent requirements that should affect the agent's behavior during the interaction, such as constraints, preferences, output conventions, or dependencies on prior context. Checklist items capture final completion requirements that can be verified from the trajectory, produced artifacts, or tool records.

Feasibility validation. After annotation, annotators validate each task before inclusion in the benchmark. This step checks whether the task is solvable with the provided context, whether the required tools and files are correctly connected, and whether the evaluation criteria can be applied reliably. The validation process includes:

- Construct a reference workflow that describes a plausible completion path using the available workspace, tools, and interaction protocol.
- Create expected artifacts when the task requires concrete outputs, such as documents, tables, code files, or structured records.
- Bind required tools, input files, and workspace paths to ensure that the agent can access the

Aspect	Do ✓	Avoid ×
Naturalness	Use concise wording that matches the user's role and working style.	Do not use artificial benchmark-like phrasing or expose internal task metadata.
Actionability	State a clear surface goal so that the agent can begin useful work.	Do not make the request so vague that the agent must ask the user to restate the task.
Underspecification	Omit at least one task-relevant requirement that the agent should infer, ask about, or recover from context.	Do not include all constraints, preferences, and output requirements in the initial request.
Environment fit	Make the request consistent with the persona, workspace state, and available tool interfaces when applicable.	Do not refer to files, records, or actions that are unavailable in the task environment.

TABLE 9 Annotation guidelines for initial requests.

information needed for completion.

- Run pilot executions to identify missing context, broken tool paths, unclear instructions, unstable graders, or checklist items that are difficult to judge.

Tasks are revised or removed when validation reveals ambiguity or infeasibility. Revisions may include rewriting the initial request, refining hidden intents, splitting checklist items, adding missing workspace context, or adjusting graders for more stable verification.

I.3 Annotation Guidelines

Initial requests. The initial request defines the user-facing starting point of a session. Annotators write it to be natural and actionable, while deliberately leaving some task-relevant requirements unstated so that proactive behavior can be evaluated. Tab. 9 summarizes the main annotation rules.

Hidden intents. Hidden intents specify unstated requirements that should shape the agent's behavior during interaction. Annotators use them to define what a proactive assistant should complete directly or elicit through a targeted question. Tab. 10 summarizes the main rules.

Checklists and graders. Checklist items define verifiable outcome criteria for completeness evaluation. Annotators write them separately from hidden intents so that proactive intent resolution and final task completion can be measured independently. Tab. 11 summarizes the main rules.

Aspect	Do ✓	Avoid ×
Unstated requirement	Each hidden intent must be absent from the initial request.	Do not restate information that is already explicitly given to the agent.
Task relevance	Each hidden intent should affect task handling, output quality, or downstream decisions.	Do not add arbitrary preferences that have no effect on the task.
Specificity	Write each intent precisely enough to support terminal status assignment during interaction.	Do not use vague intents such as "make it better" or "follow best practice" without a concrete meaning.
Scope	Mark whether the intent is session-local or persistent across sessions when relevant.	Do not let persistent intents appear without supporting evidence in prior context or workspace state.

TABLE 10 Annotation guidelines for hidden intents.

I.4 Quality Control

Before inclusion in the final benchmark, each task undergoes iterative checks and revisions. Annotators inspect the task package for leakage and recoverability, ensuring that hidden intents are not directly exposed in the initial request while still being recoverable from prior sessions, workspace artifacts, memory, tool results, or targeted clarification. They also run pilot executions to detect setup issues, including missing files, invalid tool bindings, broken workspace paths, and infeasible task flows. Finally, they validate the evaluation resources by checking that rubric-based criteria can be judged from the selected evidence and that rule-based graders behave correctly on expected cases. Tasks are revised or removed when these checks reveal ambiguity, infeasibility, or unstable evaluation.

J Case Study

J.1 Hidden Intents

Failure case: Researcher – Claude 4.6 Opus. Fig. 6 and Fig. 7 show a representative hidden-intent case from the Researcher user episode evaluated with Claude 4.6 Opus. The session begins from an environment-triggered paper recommendation feed, where the relevant requirement is not stated as a direct user instruction. The agent first handles the feed as a broad recommendation task, and only later focuses on the OpenClaw-related subset after the user explicitly asks for it. The key signal is the hidden-intent assignment: several intents are marked as *provided* rather than *completed* or *inferred*, indicating that the user had to surface requirements that the agent did not proactively resolve. Although the final trajectory has moderate completeness, this case mainly illustrates weak proactive intent resolution under an underspecified trigger.

Case Study – Researcher – Claude 4.6 Opus: Task and Trigger

▶ TASK Filter papers the user cares about most from a simulated Hugging Face daily-papers feed. The session is started by an environment event, so the agent must decide whether the event is actionable without waiting for a direct user command.

→ INITIAL REQUEST *This is an environment-triggered initial request. The event lists candidate papers, but does not explicitly state the OpenClaw filter or the link and metadata requirements.*

source=huggingface_hub; event=paper_recommendation_trigger; target=agent; papers=

paper_01: Common Corpus: The Largest Collection of Ethical Data for LLM Pre-Training;

paper_02: Q-RAG: Long Context Multi-Step Retrieval via Value-Based Embedder Training;

paper_03: From movement to cognitive maps: recurrent neural networks reveal how locomotor development shapes hippocampal spatial coding;

paper_04: FIRE: Frobenius-Isometry Reinitialization for Balancing the Stability-Plasticity Tradeoff;

paper_05: Exchangeability of GNN Representations with Applications to Graph Retrieval;

paper_06: A Trajectory-Based Safety Audit of Clawdbot (OpenClaw);

paper_07: Enhancing Generative Auto-bidding with Offline Reward Evaluation and Policy Search;

paper_08: Why DPO is a Misspecified Estimator and How to Fix It;

paper_09: WebDevJudge: Evaluating (M)LLMs as Critiques for Web Development Quality;

paper_10: SafeDPO: A Simple Approach to Direct Preference Optimization with Enhanced Safety;

paper_11: MedAgentGym: A Scalable Agentic Training Environment for Code-Centric Reasoning in Biomedical Data Science;

paper_12: Optimistic Task Inference for Behavior Foundation Models;

paper_13: CounselBench: A Large-Scale Expert Evaluation and Adversarial Benchmarking of Large Language Models in Mental Health Question Answering;

paper_14: AstaBench: Rigorous Benchmarking of AI Agents with a Scientific Research Suite;

paper_15: MetaClaw: Just Talk An Agent That Meta-Learns and Evolves in the Wild;

paper_16: Neon: Negative Extrapolation From Self-Training Improves Image Generation;

paper_17: Compositional Diffusion with Guided search for Long-Horizon Planning;

paper_18: Visual symbolic mechanisms: Emergent symbol processing in Vision Language Models;

paper_19: Addressing divergent representations from causal interventions on neural networks;

paper_20: Cross-Domain Lossy Compression via Rate- and Classification-Constrained Optimal Transport;

paper_21: Latent Fourier Transform;

paper_22: GLASS Flows: Efficient Inference for Reward Alignment of Flow and Diffusion Models;

paper_23: Latent Speech-Text Transformer;

paper_24: LoongRL: Reinforcement Learning for Advanced Reasoning over Long Contexts;

paper_25: Improving Developer Emotion Classification via LLM-Based Augmentation;

paper_26: Revisiting Multilingual Data Mixtures in Language Model Pretraining;

paper_27: One-Shot Style Personalization for RL Agents via Latent Discriminator;

paper_28: OpenClaw-RL: Train Any Agent Simply by Talking;

paper_29: Compositional HyperModules for Few-Shot Code Adaptation in Meta-Reinforcement Learning;

paper_30: All-in-One: Boosting Basic Capabilities in one Omni-MLLM to Enhance Movie Understanding;

paper_31: Contrastive Code Graph Embeddings for Reinforcement Learning-Based Automated Code Refactoring;

paper_32: Soft Non-Diagonality Penalty Enables Latent Space-Level Interpretability of pLM at No Performance Cost;

paper_33: Adaptive Mixing of Non-Invariant Information for Generalized Diffusion Policy;

paper_34: Style2Shape: Image Style Guided 3D Shape Material Generation;

paper_35: Contrastive-Aligned Knowledge Distillation for Collaborative Code Completion via Multi-Agent Reinforcement Learning;

paper_36: Scaling Laws for Generative Reward Models;

paper_37: Contrastive-Online-Meta (COM): A Dynamic Adaptation Mechanism for Instruction-Tuned CodeLLMs;

paper_38: FedPAC: Consistent Representation Learning for Federated Unsupervised Learning under Data Heterogeneity;

paper_39: Training as Computation: A Resource-Bounded Theory of Continual Self-Play Learning;

paper_40: Cross-Modal Syntax-NL Attention for Multi-Agent Reinforcement Learning in Collaborative Coding;

paper_41: Less is More: Improving Molecular Force Fields with Minimal Temporal Information;

paper_42: Curricular Adversarial Training for Robust Code Generation via Hierarchical Reinforcement Learning;

paper_43: "Humans welcome to observe": A First Look at the Agent Social Network Moltbook;

paper_44: Triple-S: A Sticker Semantic Similarity Benchmark with General Sticker Encoder;

paper_45: Unsupervised Dynamic Graph Multi-Model Representation Learning for Temporal Patterns Discovery: Uncovering Parkinson's Disease Stages Using Cerebrospinal Fluid Longitudinal Profiles;

paper_46: Causal Reasoning Favors Encoders: Limits of Decoder-Only Models;

paper_47: Hierarchies over Pixels: A Benchmark for Cognitive Geospatial Reasoning for Agents;

paper_48: But what is your honest answer? Aiding LLM-judges with honest alternatives using steering vectors;

paper_49: LaMbDA: Local Latent Embedding Alignment for Cross-modal Time-Series Diffusion.

★ HIDDEN INTENTS

∘ **I1** List the OpenClaw-related papers from the recommendation trigger.

Case Study – Researcher – Claude 4.6 Opus: Trajectory and Scores

→ INPUT **Turn 1.** Environment trigger with the 49-paper payload in Fig. 6.

⇐ OUTPUT SUMMARY **Turn 1.** The agent filters the batch by broad interests: "think-with-image," multimodal reasoning, multimodal tool use, and VLM/MLLM agents. It highlights several adjacent multimodal papers, but does not surface the annotated OpenClaw-related subset.

→ INPUT **Turn 2.** *Give list OpenClaw-related papers from this recommendation trigger.* **provided: I1**

⇐ OUTPUT **Turn 2.** *From the recommendation batch, here are the OpenClaw-related papers: A Trajectory-Based Safety Audit of Clawdbot (OpenClaw); MetaClaw: Just Talk An Agent That Meta-Learns and Evolves in the Wild; OpenClaw-RL: Train Any Agent Simply by Talking. That's 3 papers referencing the OpenClaw ecosystem.*

→ INPUT **Turn 3.** *Give add a short introduction and key technical points for each OpenClaw-related paper.* **provided: I2**

⇐ OUTPUT SUMMARY **Turn 3.** The agent searches web sources, fetches arXiv pages and a project page, and writes introductions plus technical points. It also includes several arXiv and GitHub links. **completed: I3** **completed: I4**

→ INPUT **Turn 4.** *Give the research institution and author for each OpenClaw-related paper.* **provided: I5**

⇐ OUTPUT SUMMARY **Turn 4.** The agent searches for author and affiliation metadata and then reports institutions and authors for the three selected papers.

▶ EVALUATION **Comp=65.00** **Proc=40.00**

★ CHECKLIST SIGNALS Satisfied: several paper recommendations, selected links, and selected author/institution fields. Not satisfied: full annotated paper coverage and some detailed metadata criteria.

FIGURE 7 Trajectory and scores; colored intent tags indicate terminal status, e.g., **provided: I1** denotes a user-provided intent and **completed: I3** denotes an agent-completed intent.

Aspect	Do ✓	Avoid ×
Verifiability	Each item should be checkable from the trajectory, produced artifacts, or tool records.	Do not include criteria that require unstated assumptions or external judgment beyond the available evidence.
Atomicity	Each item should cover one requirement whenever possible.	Do not combine several independent requirements into one checklist item.
Outcome focus	Write checklist items as final completion conditions.	Do not use checklist items to score whether the agent was proactive during the interaction.
Grader choice	Use rubric-based evaluation for semantic criteria and rule-based verification for exact structured criteria.	Do not use an LLM rubric when an exact programmatic check is available and reliable.
Auditability	Make the expected evidence clear enough for another expert to inspect the judgment.	Do not write criteria whose satisfaction cannot be traced to concrete evidence.

TABLE 11 Annotation guidelines for checklist items and grader assignment.

Contrast case: Researcher – DeepSeek V3.2 and Claude 4.6 Opus. Fig. 8 and Fig. 9 show the DeepSeek V3.2 trace, while Fig. 10 and Fig. 11 show the Claude 4.6 Opus trace for the same Researcher meal-planning task. Both runs receive the same initial request, which explicitly mentions only a one-week meal plan and the RMB 20–30 price constraint, while the user's body profile, macro accounting, table structure, and Plan B requirements remain hidden but recoverable from prior context. We focus on the first assistant turn because it occurs before the user reveals any hidden requirements and therefore best isolates proactive recovery from visible-request following. DeepSeek V3.2 mostly follows the visible budget and availability cue in this first response; although it recognizes campus-accessible food options, it does not recover the prior-session body profile, table preference, per-meal macro requirement, or Plan B preference, so the user has to reveal several requirements across later turns. Claude 4.6 Opus captures more task structure immediately: it goes beyond the visible price constraint by imposing a readable table, listing concrete foods for each day and meal, and grounding choices in campus or nearby meal availability. However, Claude still requires later user turns for macro totals, per-meal P/F/C values, and fallback meals. The contrast is a stronger-versus-weaker proactivity comparison rather than a binary success-failure case: both trajectories improve after user intervention, but their scores differ according to how much hidden intent is resolved before the user supplies it.

Success case: Marketer – GPT-5.4. Fig. 12 and Fig. 13 show a targeted-followup success case from a Marketer crisis-communication episode evaluated with GPT-5.4. The initial trigger is a system webhook requesting a final X apology letter after a prior client-alignment session, but

Case Study – Researcher – DeepSeek V3.2: Task and Hidden Intents

▶ TASK Design a one-week meal plan with per-meal prices controlled within RMB 20–30. The visible request only asks for a meal plan and prices, while the full task also requires body-profile conditioning, macro accounting, table structure, obtainable foods, and fallback meals.

▶ PRIOR CONTEXT In a prior muscle-gain gear-selection session, the user had already exposed the same body profile, the muscle-gain goal, and preferences for readable tables and Plan B alternatives.

→ INITIAL REQUEST

> Help me design a one-week meal plan. Provide the price of each meal, controlled within RMB 20–30.

★ PARTIAL HIDDEN INTENTS

- **I1** Use the user's body profile: about 175 cm, 68 kg, 17% body fat, and muscle-gain phase.
- **I2** Provide daily totals for protein, fat, and carbohydrates in grams.
- **I3** Use a readable table format.
- **I4** List specific foods for each meal on each day.
- **I5** Prefer meals obtainable from school cafeterias, nearby shops, or common takeout platforms.
- **I6** Give per-meal protein, fat, and carbohydrate amounts in grams.
- **I7** Provide a substitute option, Plan B, for each meal.

FIGURE 8 Task trigger and partial hidden intents.

it does not restate the approved engineering cause, compensation language, or full incident scale. Instead of fabricating a publishable statement from incomplete context, the agent asks focused questions about the missing approval facts. The user-agent tracking marks the matched targeted-followup indexes for the engineering explanation and compensation as *inferred*, meaning the agent's clarification directly elicited hidden intents that would otherwise have remained unstated. After the user supplies those two requirements, the next response states the approved cause and compensation while also incorporating the incident-scale facts recovered from the prior context, reaching perfect Comp and Proc. This case illustrates a successful proactive pattern: a targeted clarification can count as intent inference when it asks for the specific missing fact needed to complete a high-stakes artifact.

Success case: Pharmacist – Kimi K2.5. Fig. 14 and Fig. 15 show a targeted-followup case from a Pharmacist sandbox-Gmail workflow evaluated with Kimi K2.5. The initial request asks the agent to read a local booking brief, send a simulated Gmail message, and verify the send, but the available tool set reported by the agent does not expose Gmail functions. In the first turn, the agent asks a focused question about which email-sending method to use, and the user-agent marks the direct-send workflow intent as *inferred*. The same response also recovers key brief-derived message requirements by drafting an instrument-availability request with Tuesday/Thursday afternoon availability, the 4-hour LC-MS slot, and the 30-minute sample-preparation note, although later checklist scoring still penalizes exact block content and tool evidence. This case is useful because it separates hidden-intent proactivity from final task success:

Case Study – Researcher – DeepSeek V3.2: Trajectory and Scores

→ INPUT **Turn 1.** Initial request in Fig. 8.

⇐ OUTPUT SUMMARY **Turn 1.** The agent creates a loose day-by-day meal plan with budget notes, Fudan/campus pricing assumptions, and availability-oriented cost-saving tips. It does not yet use the body profile, daily macro targets, per-meal macro values, a compact readable meal-plan table, or per-meal Plan B substitutes. **completed: I5**

→ INPUT **Turn 2.** *Height about 175 cm, weight about 68 kg, body-fat ratio about 17%; currently in a muscle-gain phase.* **provided: I1**

⇐ OUTPUT SUMMARY **Turn 2.** The agent revises the plan for muscle gain, mentions higher protein intake and calorie surplus, and reports rough protein totals, but still does not produce the required daily macro totals, per-meal macro table, or Plan B structure.

→ INPUT **Turn 3.** *Based on the user's body weight, provide the required daily totals of the three macronutrients (protein/fat/carbohydrates) for each day of the week, in grams (g).* **provided: I2**

⇐ OUTPUT **Turn 3.** *Based on your body weight (68kg) and muscle-gain goals, I'll calculate the daily macronutrient requirements for each day of the week.* The response then gives protein, carbohydrate, and fat targets, distinguishing training from rest days.

→ INPUT **Turn 4.** *For each meal, indicate macronutrient amounts (protein/fat/carbohydrates), in grams (g).* **provided: I6**

⇐ OUTPUT SUMMARY **Turn 4.** The agent provides detailed per-meal macro breakdowns for the revised plan, but the response remains verbose and does not yet include the required Plan B substitutes.

→ INPUT **Turn 5.** *Provide a substitute option (Plan B) for each meal to handle unexpected situations.* **provided: I7**

⇐ OUTPUT **Turn 5.** *I'll provide substitute options (Plan B) for each meal to handle unexpected situations like campus canteen closures, food availability issues, or budget constraints.* The response then adds substitute meals and emergency backup options after the user explicitly asks for Plan B.

▶ EVALUATION **Comp=87.50** **Proc=42.86**

★ CHECKLIST SIGNALS Final satisfied: C1, C2, C3, C4, C6, C7, C8. Not satisfied: C5, the final table does not reliably list the specific foods for each meal each day.

FIGURE 9 DeepSeek trajectory and scores.

Case Study – Researcher – Claude 4.6 Opus: Task and Hidden Intents

▶ TASK Design a one-week meal plan with per-meal prices controlled within RMB 20–30. The visible request only asks for a meal plan and prices, while the full task also requires body-profile conditioning, macro accounting, table structure, obtainable foods, and fallback meals.

▶ PRIOR CONTEXT In a prior muscle-gain gear-selection session, the user had already exposed the same body profile, the muscle-gain goal, and preferences for readable tables and Plan B alternatives.

→ INITIAL REQUEST

> Help me design a one-week meal plan. Provide the price of each meal, controlled within RMB 20–30.

★ PARTIAL HIDDEN INTENTS

- **I1** Use the user's body profile: about 175 cm, 68 kg, 17% body fat, and muscle-gain phase.
- **I2** Provide daily totals for protein, fat, and carbohydrates in grams.
- **I3** Use a readable table format.
- **I4** List specific foods for each meal on each day.
- **I5** Prefer meals obtainable from school cafeterias, nearby shops, or common takeout platforms.
- **I6** Give per-meal protein, fat, and carbohydrate amounts in grams.
- **I7** Provide a substitute option, Plan B, for each meal.

FIGURE 10 Task trigger and partial hidden intents.

the targeted follow-up and partial brief-detail recovery raise PROC, while the final checklist remains low because the Gmail login, direct send, and outbox confirmation are not supported by tool evidence.

J.2 Checklist

Success case: Law Trainee – GPT-5.4. Fig. 16 shows the shared task and checklist targets, and Fig. 17 shows the higher-scoring GPT-5.4 trajectory from the Law Trainee user episode. The GPT-5.4 run shows a high-completeness checklist pattern: after the missing handover details are surfaced during interaction, the agent sends a consolidated SMS that covers the pickup slot, required originals, reply deadline, and risk-flag instruction, while also creating and deleting the temporary Todoist board with the required reminder content. The remaining misses are mainly strict tool-string checks, so this case illustrates how checklist evaluation can separate strong end-task completion from exact tool-evidence failures.

Failure case: Law Trainee – Kimi K2.5. Fig. 18 shows the lower-scoring Kimi K2.5 trajectory on the same Yuehai court-side handover task. The Kimi K2.5 trajectory completes some visible actions, including sending SMS messages and briefly using Todoist, but it leaves several checklist requirements unsupported by the final trace. The handover information is split across messages rather than preserved as one compact phone-readable SMS, the reply deadline and risk-flag instruction are absent, and the Todoist reminder content does not cover the annotated 10:45

Case Study – Researcher – Claude 4.6 Opus: Trajectory and Scores

→ INPUT **Turn 1.** Initial request in Fig. 10.

⇐ OUTPUT **Turn 1.** *Here's a practical meal plan designed for a student in Shanghai, with each meal priced between RMB 20–30. Since you're on a muscle-gain program, I've emphasized high protein where possible while keeping costs down.* The response then gives a readable seven-day table with concrete meals and prices, but does not yet provide daily macro targets, per-meal macro values, or Plan B. **completed: I3** **completed: I4** **completed: I5**

→ INPUT **Turn 2.** *For each meal, indicate macronutrient amounts (protein/fat/carbohydrates), in grams (g).* **provided: I6**

⇐ OUTPUT SUMMARY **Turn 2.** The agent adds protein, fat, carbohydrate, and calorie columns for each meal and states that the plan is designed for a 175 cm, 68 kg student on a muscle-gain program.

→ INPUT **Turn 3.** *Based on the user's body weight, provide the required daily totals of the three macronutrients (protein/fat/carbohydrates) for each day of the week, in grams (g).* **provided: I2**

⇐ OUTPUT **Turn 3.** *Your daily macro target is constant at 130g protein / 60g fat / 325g carbs regardless of the day.* The response computes this lean-bulk target from 68 kg and about 17% body fat, then gives a daily target table across the week.

→ INPUT **Turn 4.** *Provide a substitute option (Plan B) for each meal to handle unexpected situations.* **provided: I7**

⇐ OUTPUT SUMMARY **Turn 4.** The agent produces a complete Plan A/Plan B table with per-meal P/F/C values and prices, then adds snack guidance to bridge the gap between meals and the lean-bulk targets.

▶ EVALUATION **Comp=100.00** **Proc=71.43**

★ CHECKLIST SIGNALS Final satisfied: C1, C2, C3, C4, C5, C6, C7, C8. The final checklist score is perfect, while several hidden intents are still marked as user-provided in the trajectory above.

FIGURE 11 Claude trajectory and scores.

Case Study – Marketer – GPT-5.4: Task and Hidden Intents

▶ TASK Generate the final X apology letter for MeowConnect after a crisis-management approval phase. The visible trigger says the final letter is required, but the concrete incident facts depend on a prior client-alignment session.

→ INITIAL REQUEST

```
[INCOMING SYSTEM WEBHOOK: CRISIS_MANAGEMENT_PLATFORM]
client=MeowConnect; status=PHASE_1_ALIGNMENT_COMPLETE;
next_action_required=GENERATE_AND_PUBLISH_FINAL_APOLOGY_LETTER;
target_platform=X_Twitter; context_dependency=PRIOR_CLIENT_ALIGNMENT_SESSION;
output_directive=STDOUT_RAW_TEXT_ONLY
```

★ HIDDEN INTENTS

- **I1** Integrate the exact incident scale: downtime duration and number of affected active nodes.
- **I2** State the official engineering explanation.
- **I3** Announce the approved compensation.

FIGURE 12 Task trigger and hidden intents.

Case Study – Marketer – GPT-5.4: Targeted Follow-up and Scores

→ INPUT **Turn 1.** System-webhook trigger in Fig. 12.

⇐ OUTPUT **Turn 1.** *We're at the stage where the final apology letter is required, but I'm still missing several facts from the prior alignment session that materially affect what can be safely published on X.* The agent asks a focused clarification list, including the approved remediation and legal/cause wording beyond "unforeseen extreme fluctuations in cloud services." **inferred: I2** **inferred: I3**

→ INPUT **Turn 2.** *The letter explicitly states the official engineering explanation. The letter explicitly announces the compensation.*

⇐ OUTPUT SUMMARY **Turn 2.** The agent writes the final apology letter with the May 1–3 outage window, approximately 150,000 active nodes, the cloud-services engineering cause, and the $1 Million Fur Baby Medical Reimbursement Fund. **completed: I1**

▶ EVALUATION **Comp=100.00** **Proc=100.00**

★ CHECKLIST SIGNALS Final satisfied: C1, C2, C3, C4, C5. Not satisfied: none.

FIGURE 13 GPT-5.4 trajectory and scores.

Case Study – Pharmacist – Kimi K2.5: Task and Hidden Intents

▶ TASK Send a sandbox Gmail follow-up for an LC-MS instrument booking request, then verify the send through the outbox or matching thread. The visible request gives the synthetic login credentials and final response format, while the hidden requirements specify the correct Gmail workflow and brief-derived message details.

↦ INITIAL REQUEST SUMMARY The user asks the agent to read the local instrument booking brief, log into sandbox Gmail with synthetic credentials, send the simulated message rather than stopping at a draft, confirm the send through the outbox or matching thread, and return exactly three final blocks.

★ PARTIAL HIDDEN INTENTS

- **I2** Use Gmail's direct send-email flow instead of relying only on draft creation.
- **I4** Include both the request to confirm instrument availability and the Tuesday/Thursday afternoon availability sentence.
- **I5** Include the 4-hour LC-MS slot request and the note that sample prep can be ready 30 minutes before the run.

FIGURE 14 Task trigger and partial hidden intents.

Case Study – Pharmacist – Kimi K2.5: Targeted Follow-up and Scores

→ INPUT **Turn 1.** Initial request in Fig. 14.

⇐ OUTPUT **Turn 1.** *Could you clarify: Is there a Gmail app tool I should be using that might be named differently? Should I use the message tool instead to simulate the email send?* The focused question directly targets the direct-send workflow. The same response drafts the booking email with the availability request, approximate Tuesday/Thursday afternoon windows, the 4-hour LC-MS slot, and the 30-minute sample-prep note. **inferred: I2** **completed: I4** **completed: I5**

→ INPUT **Turn 2.** *Use Gmail's direct send-email flow to send a new email instead of relying only on draft creation.*

⇐ OUTPUT SUMMARY **Turn 2.** The agent repeatedly reports that Gmail tools are unavailable and eventually produces a simulated final response through the chat channel. The required Gmail login, direct send, exact brief block content, and outbox/thread verification remain unsupported by checklist or tool evidence.

▶ EVALUATION **Comp=11.11** **Proc=42.86**

★ CHECKLIST SIGNALS Final satisfied: C4. Not satisfied: C1, C2, C3, C5, C6, C7, C8, C9.

FIGURE 15 Kimi K2.5 trajectory and scores.

packet-completeness and stamped-page checks. This contrast highlights why checklist-based grading is necessary in addition to surface-level task progress: both agents interact with the same tools, but only the higher-scoring trajectory preserves the operational details needed for a reliable handover.

Case Study – Law Trainee – GPT-5.4 / Kimi K2.5: Task and Checklist

▶ **TASK** Coordinate a court-side handover for the Yuehai guarantee packet by sending one compact SMS and running a temporary Todoist reminder pass. The task requires both user-facing message quality and exact tool-side cleanup.

→ **INITIAL REQUEST** *Credentials and phone identifiers are omitted here; the trace contains the required phone and Todoist access information.*

> The user asks the agent to send one compact SMS for a Yuehai guarantee packet court-side handover, create a temporary Todoist board named "Temporary Court Handover Reminder Board," add one short reminder item covering two pre-handover checks, and remove the temporary board before replying.

★ **CHECKLIST TARGETS**

C1 SMS should identify the Yuehai guarantee dispute court-side handover.
C2 SMS should confirm 11:30 a.m. tomorrow at the east gate of Jiangzhou Intermediate People's Court.
C3 SMS should list the original notarized power of attorney, two stamped evidence-list copies, and one hearing-notice copy.
C4 SMS should include a concrete reply deadline before 8:00 p.m. today.
C5 SMS should ask the recipient to flag immediately if documents are not ready or the pickup time or location needs to change.
C6 At least one SMS should present the handover slot, packet items, reply-by instruction, and risk-flag instruction in a compact phone-readable format.
C7 Todoist reminder coverage should include a 10:45 a.m. packet-completeness recheck.
C8 Todoist reminder coverage should include a stamped-page check.
C9 The temporary Todoist board should be removed by the end of the task.

In addition to C1–C9, the checklist also includes code-based tool evaluation that inspects the actual SMS payload and Todoist create/delete calls.

FIGURE 16 Task overview and checklist targets.

J.3 Cross-Session Dependency Design

Success case: Researcher – Claude 4.6 Opus. Fig. 19 shows a cross-session dependency case from the Researcher user episode, where the current request refers only to the user's research theme. The relevant theme and output conventions were established in earlier sessions: the user had explored think-with-image multimodal agents, preferred a small batch of papers, and expected links, open-source status, and concrete follow-up paths. As shown in Fig. 20, Claude 4.6 Opus carries over the research theme and several output conventions, allowing it to complete much of the target paper-selection and metadata checklist within three turns.

Case Study – Law Trainee – GPT-5.4: Trajectory and Scores

⇒ GPT-5.4 INPUT **Turn 1.** Initial request for one compact Yuehai handover SMS and a temporary Todoist reminder board that must be cleared before reply.

⇐ GPT-5.4 OUTPUT SUMMARY **Turn 1.** The agent logs into the phone and Todoist tools, but asks for the missing handover details and two pre-handover checks before sending the SMS or creating the final reminder item.

⇒ GPT-5.4 INPUT **Turn 2.** *In the SMS, confirm pickup for 11:30 a.m. tomorrow at the east gate of Jiangzhou Intermediate People's Court.*

⇐ GPT-5.4 OUTPUT SUMMARY **Turn 2.** The agent sends an SMS with the Yuehai packet, the 11:30 a.m. pickup, and the court east gate. It also creates a temporary Todoist board with a placeholder pre-handover reminder and deletes the board.

⇒ GPT-5.4 INPUT **Turn 3.** *Put the handover details into one compact SMS that is easy to read on a phone, instead of splitting the handover across several texts.*

⇐ GPT-5.4 OUTPUT **Turn 3.**

> I can draft it as one compact SMS, but I need the actual handover details to include.
>
> Send me the essentials and I'll condense them into a single phone-friendly text:
>
> - time
> - place
> - recipient/contact
> - packet/material name
> - any check-in phrase or fallback instruction
>
> Clarify: Please paste the handover details you want included in the single SMS.

⇒ GPT-5.4 INPUT **Turn 4.** *In the SMS, ask them to bring the original notarized power of attorney, two stamped copies of the evidence list, and one copy of the hearing notice.*

⇐ GPT-5.4 OUTPUT SUMMARY **Turn 4.** The agent sends a follow-up SMS that combines the pickup slot with the three required originals, but the reply deadline, risk-flag instruction, and specific reminder checks are still absent.

⇒ GPT-5.4 INPUT **Turn 5.** *In the SMS, ask for a reply before 8:00 p.m. today and ask them to flag immediately if any document is not ready or if the pickup time or location needs to change. In the temporary Todoist board, include a 10:45 a.m. packet-completeness recheck and a stamped-page check.*

⇐ GPT-5.4 OUTPUT SUMMARY **Turn 5.** The agent sends an updated compact SMS covering the pickup slot, required originals, reply deadline, and risk-flag instruction. It then creates the temporary Todoist board, adds a reminder for the 10:45 packet-completeness and stamped-page checks, and deletes the board before replying.

▶ GPT-5.4 EVALUATION **Comp=83.33** **Proc=40.00**

★ GPT-5.4 CHECKLIST SIGNALS Satisfied: all semantic SMS checklist items, both Todoist reminder-content items, correct temporary-board cleanup, and most tool-side checks. Not satisfied: several strict tool-string checks for the SMS payload.

FIGURE 17 GPT-5.4 trajectory and scores.

Case Study – Law Trainee – Kimi K2.5: Trajectory and Scores

→ KIMI K2.5 INPUT **Turn 1.** Same initial request for one compact Yuehai handover SMS and a temporary Todoist reminder board that must be cleared before reply.

⇐ KIMI K2.5 OUTPUT SUMMARY **Turn 1.** The agent sends a generic Yuehai guarantee packet message, creates the temporary Todoist board, adds generic ID and case-number checks, and deletes the board. The message does not include the exact pickup slot, required originals, reply deadline, or risk-flag instruction.

→ KIMI K2.5 INPUT **Turn 2.** *In the SMS, confirm pickup for 11:30 a.m. tomorrow at the east gate of Jiangzhou Intermediate People's Court.*

⇐ KIMI K2.5 OUTPUT **Turn 2.**

> Pickup confirmation sent!
>
> **Message to 2873148336:**
>
> - Tomorrow (Saturday) at 11:30 a.m.
> - East gate, Jiangzhou Intermediate People's Court
> - Yuehai guarantee packet
>
> All details confirmed – you're covered for the handover.
>
> **Complete**

→ KIMI K2.5 INPUT **Turn 3.** *In the SMS, ask them to bring the original notarized power of attorney, two stamped copies of the evidence list, and one copy of the hearing notice.*

⇐ KIMI K2.5 OUTPUT SUMMARY **Turn 3.** The agent sends another separate SMS listing the three required originals. The final trace still lacks the reply-before-8 p.m. deadline, the immediate risk-flag instruction, and the required Todoist reminder checks.

▶ KIMI K2.5 EVALUATION **Comp=38.89** **Proc=40.00**

★ KIMI K2.5 CHECKLIST SIGNALS Satisfied: SMS was sent to the correct recipient, some pickup/material information appears across the trace, and the temporary board is created and removed. Not satisfied: one-message compactness, reply deadline, risk-flag instruction, required Todoist reminder coverage, and strict SMS payload checks.

FIGURE 18 Kimi K2.5 trajectory and scores.

Failure case: Researcher – Kimi K2.5. Fig. 21 shows the lower-scoring Kimi K2.5 trajectory on the same task. Instead of recovering the user's research theme and presentation preferences from prior sessions, the agent repeatedly waits for the user to restate them. The resulting conversation is longer, and the selected papers drift away from the annotated target set, leading to low proactivity and zero checklist completion despite several rounds of user-provided requirements.

Case Study – Researcher – Cross-Session Dependency: Task and Hidden Intents

▶ **TASK** Organize accepted ICLR papers from a local list for the user's research theme. The current request does not restate the theme or output conventions, so the agent must recover them from earlier sessions.

→ **INITIAL REQUEST**

> I wrote a local file at `paper_list.txt` that contains some accepted ICLR 2026 papers. Please organize several papers related to my research theme for me.

→ **CROSS-SESSION SETUP**

Earlier tool-use reading task The user expects paper recommendations to include introductions, key technical points, GitHub or open-source status, and concrete follow-up paths.

Earlier think-with-image task The user focuses on think-with-image multimodal agents, prefers only a small batch of papers, and expects OpenReview links when available.

★ **HIDDEN INTENTS**

- **I1** Primarily recommend think-with-image papers.
- **I2** Recommend around five papers.
- **I3** Provide a brief introduction and key technical points for each paper.
- **I4** Provide the OpenReview link for each paper.
- **I5** Provide the GitHub link for each paper, or explicitly state that no public repository is found.
- **I6** For every paper, state whether it is suitable to follow and how to follow it.

FIGURE 19 Cross-session setup and hidden intents.

Case Study – Researcher – Claude 4.6 Opus: Trajectory and Scores

⇒ INPUT **Turn 1.** The user asks the agent to organize papers from a local ICLR accepted-paper list related to "my research theme."

⇐ OUTPUT SUMMARY **Turn 1.** The agent reads the local paper list, uses memory to identify the research theme as think-with-image, and starts filtering ICLR papers around image-driven reasoning and multimodal tool use. **completed: I1**

⇒ INPUT **Turn 2.** *Recommend around 5 papers.* **provided: I2**

⇐ OUTPUT SUMMARY **Turn 2.** The agent gives a five-paper list centered on the annotated target set, including technical summaries, OpenReview links, and GitHub or open-source status. **completed: I3** **completed: I4** **completed: I5**

⇒ INPUT **Turn 3.** *For every paper, state whether it is suitable to follow and how to follow it.* **provided: I6**

⇐ OUTPUT SUMMARY **Turn 3.** The agent adds feasibility assessments and concrete follow-up paths for the selected papers, while also considering the user's research setup and compute constraints.

▶ EVALUATION **Comp=64.00** **Proc=66.67**

★ CHECKLIST SIGNALS Satisfied: multiple target papers, links, GitHub fields, and follow-up judgments. Not satisfied: some detailed technical-point and target-paper metadata criteria.

FIGURE 20 Claude 4.6 Opus trajectory and scores.

Case Study – Researcher – Kimi K2.5: Trajectory and Scores

→ INPUT **Turn 1.** Same request to organize ICLR papers related to "my research theme."

⇐ OUTPUT SUMMARY **Turn 1.** The agent reads the paper list but asks the user to specify the research theme instead of carrying it over from earlier sessions.

→ INPUT **Turn 2.** *Primarily recommend think-with-image papers.* **provided: I1**

⇐ OUTPUT SUMMARY **Turn 2.** The agent lists broad think-with-image categories and many candidate titles, then the user has to restate the preferred batch size.

→ INPUT **Turn 3.** *Recommend around 5 papers.* **provided: I2**

⇐ OUTPUT SUMMARY **Turn 3.** The agent selects five papers, but the set drifts away from the annotated target papers and lacks the required technical details.

→ INPUT **Turn 4.** *Provide a brief introduction and key technical points for each paper.* **provided: I3**

⇐ OUTPUT SUMMARY **Turn 4.** The agent adds introductions and key points for the selected set, but the required OpenReview and GitHub evidence remains missing.

→ INPUT **Turn 5.** *Provide the OpenReview link for each paper.* **provided: I4**

⇐ OUTPUT SUMMARY **Turn 5.** The agent says it cannot access the OpenReview links from the local list and asks for more information.

→ INPUT **Turn 6.** *Provide the GitHub link for each paper; if a paper has no open-source GitHub repository, explicitly state that.* **provided: I5**

⇐ OUTPUT SUMMARY **Turn 6.** The agent searches for GitHub repositories for the selected papers, but the selected set remains off-target.

→ INPUT **Turn 7.** *For every paper, state whether it is suitable to follow and how to follow it.* **provided: I6**

⇐ OUTPUT SUMMARY **Turn 7.** The agent provides suitability advice for the wrong paper set, so the final trajectory does not satisfy the target checklist.

▶ EVALUATION **Comp=0.00** **Proc=0.00**

★ CHECKLIST SIGNALS Satisfied: none of the annotated target-paper checklist criteria. Not satisfied: target paper selection, target technical points, OpenReview links, GitHub links, and follow-up judgments.

FIGURE 21 Kimi K2.5 trajectory and scores.